Universitat de Lleida

TREBALL FINAL DE GRAU



ESCOLA
POLITÈCNICA SUPERIOR
UNIVERSITAT DE LLEIDA
INSPIRING THE FUTURE

Estudiant: **Xiao Long Ji**

Titulació: Grau en Enginyeria Informàtica

Títol de Treball Final de Grau: **Comparativa de Systemd amb Kubernetes a l'entorn Edge**

Director/a: **Oriol Agost Batalla, Jordi Mateo Fornés**

Presentació

Mes: Setembre

Any: 2026

Agraïments

Vull expressar el meu sincer agraïment als meus tutors pel seu guiatge inestimable, paciència i dedicació al llarg de tot el desenvolupament d'aquest projecte. La seva expertesa i els seus consells constants han estat fonamentals per donar forma a aquesta recerca.

També voldria donar les gràcies a la meva família pel seu suport incondicional i acompanyament en tot moment.

Resum Executiu

En l'àmbit de l'*Edge Computing*, els dispositius perimetrals solen disposar de recursos molt limitats. El desplegament d'orquestradors de contenidors moderns genera una reserva permanent de memòria i CPU, fet que redueix el marge disponible per a processar càrregues de treball exigents, com ara models d'Intel·ligència Artificial.

L'objectiu principal d'aquest treball és analitzar l'impacte sobre el rendiment que introdueix Kubernetes en entorns de maquinari limitat. Per fer-ho, es compara aquest consum de recursos amb el d'una arquitectura nativa basada en serveis de Systemd, amb la finalitat de determinar quina solució resulta més eficient en cadascun dels escenaris analitzats.

Pel que fa a la metodologia, s'han dissenyat i executat proves d'estrès automatitzades per a ambdues arquitectures utilitzant protocols de comunicació com HTTP-JSON, gRPC-Protobuf i ZeroMQ. L'avaluació s'ha centrat a mesurar el consum de memòria RAM en repòs, la degradació del rendiment de xarxa (*throughput*) i la capacitat d'autorecuperació del sistema. Per a aquesta última, la capacitat d'autorecuperació s'ha avaluat provocant l'eliminació abrupta de la unitat d'execució mitjançant els mecanismes corresponents de cada plataforma (`podman kill` en Systemd i eliminació forçada del *Pod* en K3s) per tal de calcular el temps mitjà de recuperació (*Mean Time To Repair*, MTTR).

Els resultats indiquen que Kubernetes penalitza notablement els recursos disponibles del node perimetral. L'orquestrador incrementa l'ús base de memòria en un 15 % (aproximadament 834 MB enfront dels 722 MB de Systemd) per la necessitat de mantenir actiu el seu pla de control (control plane). Així mateix, la seva capa de xarxa virtual (Software-Defined Networking, SDN) redueix la capacitat de processament en un 31 % —rebaixant el flux a unes 6.269 img/s enfront de les més de 9.053 img/s assolides de manera nativa—. Pel que fa a l'autorecuperació, la gestió directa a nivell de sistema operatiu de Systemd aconsegueix reiniciar els serveis afectats més del triple de ràpid (~452 ms) que el model distribuït de Kubernetes (~1.487 ms).

A tall de conclusió, Systemd es postula com l'opció més eficient entre les avaluades per a nodes *Edge* aïllats, on el rendiment brut i la baixa latència són crítics. El consum addicional de recursos que imposa Kubernetes només es justifica en entorns on es prioritza la simplicitat de gestió, com ara flotes distribuïdes que requereixen desplegaments declaratius i una orquestració centralitzada.

Índex

Índex de figures

Índex de taules

Capítol 1

Introducció

Aquest Treball de Fi de Grau (TFG) planteja una anàlisi comparativa d'orquestració de contenidors en entorns *Edge*: s'hi contrasta una solució d'execució nativa basada en Systemd amb un orquestrador estàndard com Kubernetes (en la seva variant lleugera, K3s).

1.1 Motivació

Els dispositius perimetrals acostumen a operar amb restriccions de maquinari notables. Aquesta limitació topa amb el desplegament d'orquestradors moderns com Kubernetes, els quals requereixen l'execució permanent de processos secundaris —com ara el *kubelet* o els components de xarxa— per garantir el seu funcionament. Aquest consum estructural actua com un llast de memòria i CPU, reduint el marge disponible per a càrregues de treball exigents, com per exemple models d'Intel·ligència Artificial. Per tant, resulta clau explorar alternatives d'execució directa a nivell de sistema operatiu per determinar si és possible recuperar aquesta capacitat de còmput sense comprometre la fiabilitat del sistema.

1.2 Objectius

L'objectiu principal d'aquest projecte és avaluar el sobrecost de rendiment que comporta l'ús de Kubernetes (K3s) en maquinari de capacitats reduïdes quan s'hi executen tasques d'Intel·ligència Artificial (concretament, la **inferència** d'un model MNIST amb PyTorch). Aquest escenari es compara amb una arquitectura nativa gestionada mitjançant Systemd, Ansible i Podman.

Per assolir aquesta premissa, es fixen els objectius específics següents:

- Quantificar l'impacte sobre la infraestructura mitjançant l'anàlisi de paràmetres base, tals com el temps de desplegament (`T_deploy`), el consum de CPU en repòs i l'ocupació de memòria RAM base.
- Avaluar el rendiment de xarxa sotmetent ambdues arquitectures a proves d'estrès sota quatre protocols diferents: HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack.
- Analitzar la penalització de la xarxa virtual a través de mètriques com el temps total de transmissió, el *throughput*, la taxa d'èxit (%), la latència RTT, el temps de processament (`T_Proc`), els pics de recursos i la mida de la càrrega útil (*payload*).
- Determinar la capacitat d'autorecuperació davant d'errors (MTTR) mitjançant la terminació abrupta de processos, contrastant la resiliència del model natiu com la de l'orquestrador.
- Automatitzar completament tant els procediments de desplegament com l'extracció de dades mitjançant *scripts* dedicats, assegurant així la total reproductibilitat de l'experiment.

1.3 Abast

L'estudi se centra en l'avaluació de nodes *Edge* aïllats, contraposant la versió lleugera de Kubernetes (K3s) amb una integració de Podman i Systemd basada en *Quadlets*, on cada *worker* executa càrregues d'inferència del model MNIST amb PyTorch. Aquesta configuració s'acompanya de l'execució sistemàtica de proves de rendiment sobre els protocols de comunicació indicats.

L'avaluació quantitativa s'estructura en dos eixos principals: d'una banda, les mètriques d'infraestructura destinades a mesurar l'impacte base en l'arrencada i el consum de recursos (incloent-hi l'MTTR); de l'altra, les mètriques de xarxa orientades a caracteritzar l'eficiència comunicativa sota condicions de càrrega.

Queden excloses de l'abast les distribucions tradicionals de Kubernetes pròpies d'entorns *Cloud*, el disseny d'arquitectures d'alta disponibilitat (HA) per al pla de control, i la utilització d'altres motors de contenidors basats en dimoni, com ara Docker.

1.4 Estructura del document

El document s'organitza en set capítols que descriuen de manera progressiva el desenvolupament i l'avaluació del projecte:

Capítol 1 (Introducció): Introdueix el context de la recerca, la motivació, els objectius establerts i l'abast tècnic.

Capítol 2 (Marc Teòric): Recull els fonaments tecnològics sobre la gestió de contenidors, els protocols de xarxa avaluats i les eines emprades (Systemd, Ansible, Kubernetes).

Capítol 3 (Estat de l'Art): Examina les solucions actuals de la indústria davant de la gestió de càrregues de treball en dispositius *Edge* amb recursos limitats.

Capítol 4 (Metodologia): Detalla el disseny experimental, l'entorn de proves configurat i la definició formal de les mètriques monitorades.

Capítol 5 (Implementació): Descriu el procés pràctic de desplegament, la integració mitjançant *Quadlets* i l'automatització dels *scripts* de mesura.

Capítol 6 (Resultats): Presenta l'anàlisi de les dades obtingudes, contrastant el comportament d'ambdues arquitectures pel que fa al consum de recursos, rendiment de xarxa i resiliència.

Capítol 7 (Conclusions): Sintetitza el compliment dels objectius, avalua els compromisos tècnics (*trade-offs*) observats i proposa possibles línies de continuïtat per a futurs treballs.

Capítol 2

Marc Teòric

Aquest apartat estableix la base teòrica necessària per comprendre l'evolució dels contenidors i l'impacte arquitectònic que els diferents models d'orquestració exerceixen sobre l'execució de càrregues d'Intel·ligència Artificial en entorns amb recursos limitats.

2.1 Tecnologies de Virtualització i Contenidors

Els contenidors formen una tecnologia de virtualització a nivell de sistema operatiu que permet empaquetar una aplicació i totes les seves dependències dins d'una unitat estandarditzada. A diferència de les màquines virtuals, els contenidors comparteixen el nucli (*kernel*) del sistema amfitrió, fet que els dota d'una lleugeresa notable, d'un inici ràpid i d'una gran eficiència en l'ús de CPU i memòria.

Arquitectures amb dimoni i sense dimoni (*Daemonless*): Tradicionalment, motors com Docker s'han basat en un dimoni (*daemon*), un procés central en segon pla —generalment executat amb privilegis d'administrador— que gestiona tot el cicle de vida dels contenidors. Si aquest dimoni falla, s'atura la totalitat del sistema. Com a evolució tecnològica, sorgeix Podman, un motor de contenidors sense dimoni que interactua directament amb el nucli mitjançant processos independents [**?**], millorant així la seguretat, reduint el consum en repòs i integrant-se de manera natural amb el sistema operatiu.

2.2 L'Evolució de l'Orquestració i el biaix a l'*Edge*

L'orquestració s'encarrega d'automatitzar el desplegament, l'escalabilitat i la resiliència de múltiples contenidors. Històricament, l'evolució d'aquests sistemes a la indústria ha passat per tres fases clarament diferenciades:

1. **La consolidació de Kubernetes al *Cloud*:** Durant els inicis de la conteneització, diverses tecnologies (com ara Docker Swarm o Apache Mesos) van competir per liderar el sector. Cap al 2018, Kubernetes es va imposar de manera absoluta, esdevenint l'estàndard indiscutible per a la gestió de grans centres de dades (*Cloud*).

2. **El dogma d'un Kubernetes cada vegada més lleuger:** Quan va sorgir la necessitat d'estendre la conteneització cap als extrems de la xarxa (*Edge Computing*), el sector va adoptar una visió reduccionista: l'única via viable consistia a "encongir" Kubernetes. Els esforços d'enginyeria es van centrar a retallar components per encabir l'orquestrador en maquinari modest, donant lloc a distribucions com K3s o MicroK8s. Malgrat tot, aquestes solucions mantenien inalterable l'arquitectura base: un procés resident (`kubelet`) per node i complexes xarxes virtuals internes (SDN).

3. **El buit metodològic:** Aquesta tendència va consolidar una falsa dicotomia en l'àmbit pràctic:

- O bé s'assumia la penalització permanent de memòria i CPU d'un Kubernetes lleuger per mantenir actiu el clúster.
- O bé s'optava per l'ús de contenidors aïllats (*standalone*), sacrificant completament la gestió centralitzada i la capacitat d'autorecuperació davant d'errors.

2.3 L'Enfocament Natiu: Systemd, Quadlets i Ansible

Per superar aquesta limitació, s'ha impulsat una alternativa tècnica —promoguda inicialment per Red Hat— que proposa un canvi de paradigma: en comptes d'afegir un agent extern (`kubelet`) que consumeixi recursos de manera constant en un dispositiu limitat, per què no delegar aquesta funció en el gestor de processos natiu que Linux ja incorpora per defecte?

Aquí és on entra Systemd, el sistema d'inicialització estàndard de Linux, dissenyat per arrencar serveis i reiniciar-los de forma gairebé instantània si fallen. Si bé anteriorment integrar contenidors amb Systemd exigia redactar fitxers de servei complexos (*Legacy*), avui dia s'empren els *Quadlets* [**?**], una utilitat nativa de Podman que permet definir els contenidors de manera declarativa i traduir-les automàticament a serveis gestionats directament pel *kernel*.

La simulació d'un orquestrador mitjançant Ansible: Com que Systemd i Podman operen de manera aïllada a cada màquina, l'arquitectura s'estén mitjançant Ansible, una eina d'automatització sense agents. Combinant aquestes tecnologies, és possible replicar l'estabilitat d'un orquestrador sencer estalviant-se el sobrecost estructural:

- **El Pla de Control (*Control Plane / Scheduler*):** Ansible assumeix el paper central. Mitjançant seqüències d'instruccions (*Playbooks*), determina on, quan i com s'han de desplegar les aplicacions a la flota de nodes.
- **L'Agent del Node (*Kubelet*):** Systemd substitueix el `kubelet`, encarregant-se de la supervisió i l'autorecuperació local en cas de caiguda de processos.
- **El Motor d'Execució (*Container Runtime*):** Podman executa els contenidors amb independència de qualsevol dimoni de fons.
- **Els Manifestos Declaratius:** Els fitxers de configuració *Quadlet* (`.container`) substitueixen els manifestos tradicionals de Kubernetes per definir l'estat desitjat del sistema.

2.4 Transmissió de Dades i Protocols de Comunicació

En entorns perimetrals, l'eficiència en el trànsit de dades és tan crítica com la capacitat de càlcul del dispositiu.

Formats de serialització de dades: La informació s'estructura habitualment sota dos enfocaments. D'una banda, els formats basats en text pla (com ara JSON o XML), altament llegibles per a humans però llastats per capçaleres repetitives i un cost de processament elevat per a la CPU. De l'altra, els formats binaris (com MessagePack o Protocol Buffers), els quals inverteixen la legibilitat humana en favor d'una gran lleugeresa i d'una serialització extremadament ràpida a nivell de maquinari, reduint de manera notable la latència.

2.5 Protocols de comunicació avaluats:

- **HTTP/REST:** L'estàndard web clàssic, generalment combinat amb JSON. Malgrat la seva simplicitat d'implementació, resulta pesat per a un flux massiu de missatges a causa del volum de les capçaleres de xarxa.

- **gRPC:** Desenvolupat per Google, empra el protocol HTTP/2 conjuntament amb la serialització binària de Protobuf. Permet la multiplexació de peticions i ofereix un rendiment òptim en arquitectures de microserveis.
- **ZeroMQ:** Més enllà d'un protocol rígid, es tracta d'una llibreria de missatgeria d'alt rendiment i sense intermediaris (*brokerless*). Permet la comunicació asíncrona directa de memòria a memòria, essent una de les alternatives més ràpides per gestionar un volum elevat de missatges per segon en xarxes locals.

2.6 Càrregues de treball exigents: La necessitat d'optimitzar per a la IA

L'aplicació pràctica d'aquestes optimitzacions es materialitza en l'execució de models d'Intel·ligència Artificial als extrems de la xarxa. Els algorismes d'*Deep Learning* requereixen allotjar estructures de dades complexes (tensors) a la memòria RAM durant tot el procés d'inferència o entrenament. Qualsevol penalització estructural de la infraestructura (com el sobrecost d'un orquestrador) sostreu recursos de còmput crítics directament al model d'IA, degradant-ne el rendiment global.

Per quantificar empíricament aquest impacte, s'utilitza com a referència el conjunt de dades MNIST, un estàndard clàssic de reconeixement de dígits manuscrits que permet enviar un flux constant de dades estructurades als contenidors objecte d'estudi [**?**].

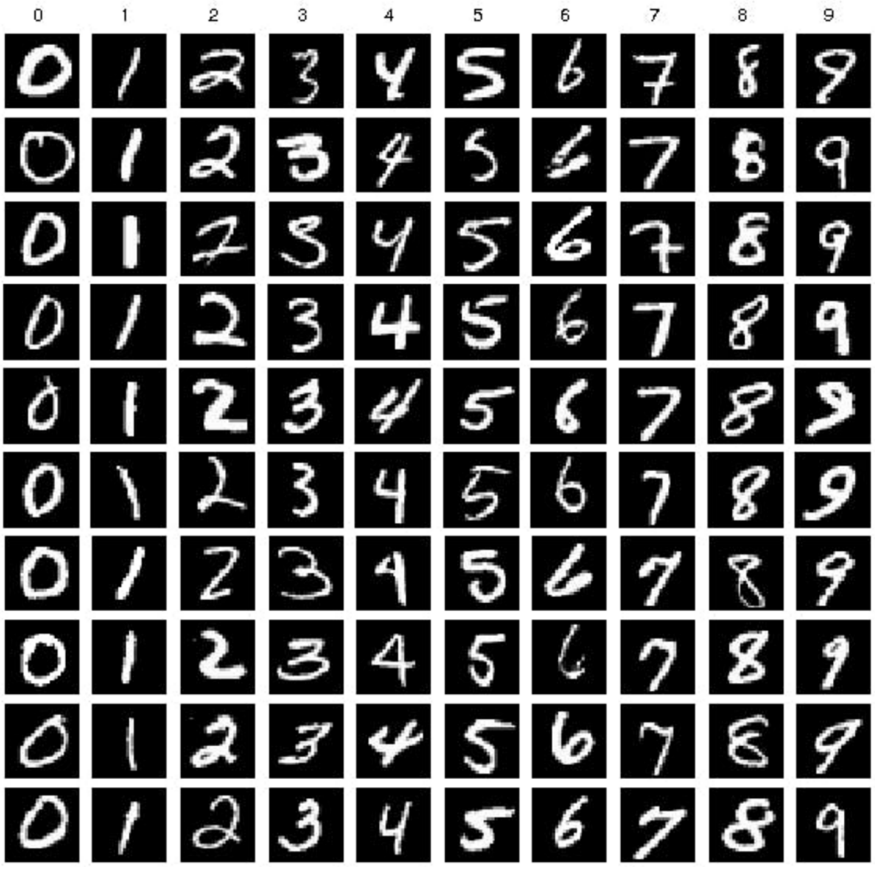


Figura 2.1: Mostres del conjunt de dades MNIST amb dígits manuscrits classificats.

Pel que fa al marc de treball d'intel·ligència artificial, tot i que TensorFlow i PyTorch dominen el mercat, s'ha seleccionat PyTorch [**?**]. La seva flexibilitat en la gestió de grafs computacionals

dinàmics proporciona una execució més àgil i un control estricte de la memòria, adaptant-se amb precisió a les limitacions dels nodes *Edge*.

Finalment, l'arquitectura de la xarxa neuronal determina el grau d'exigència computacional. En lloc d'emprar un Perceptró Multicapa (MLP) bàsic, s'ha implementat una Xarxa Neuronal Convolucional (CNN). Aquesta decisió respon a la necessitat de simular un escenari real de visió per computador: les operacions matricials pròpies d'una CNN sotmeten a un estrès real la CPU i la RAM del dispositiu, garantint que l'experiment no només avalui la xarxa, sinó que mesuri l'impacte directe de l'arquitectura sobre una càrrega de càlcul intensiva.

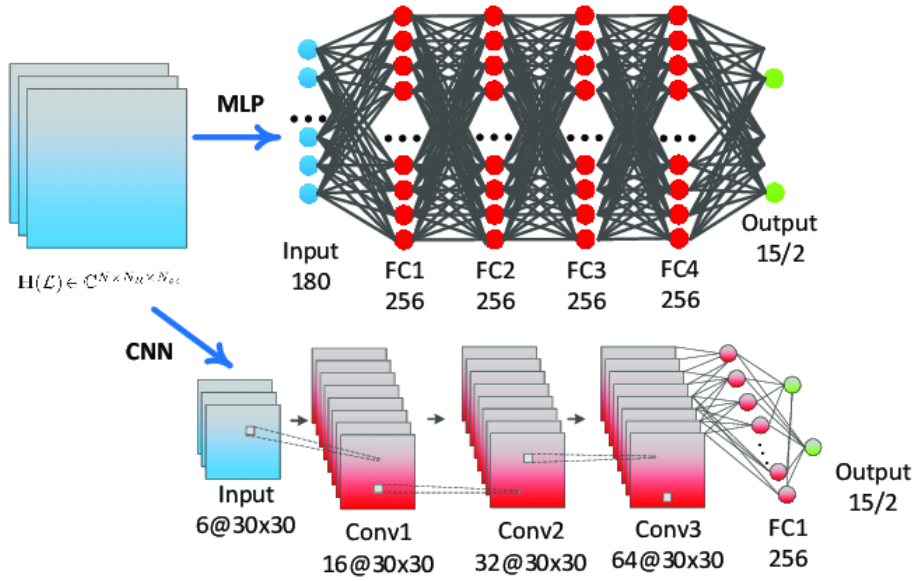


Figura 2.2: Arquitectura comparativa de xarxes MLP i CNN dissenyades per a tasques de classificació i regressió. Font: Adaptada de [**?**].

La Figura **??** il·lustra de manera conceptual l'estructura general d'una xarxa MLP i d'una CNN; les dimensions concretes hi són merament il·lustratives i no es corresponen amb l'arquitectura específica implementada en aquest treball, detallada al Capítol **??** (Listing **??**).

Capítol 3

Estat de l'Art i Treballs Relacionats

A continuació s'exposa la revisió de l'estat de l'art organitzada segons les tendències actuals en la gestió de contenidors a l'Edge Computing, analitzant-ne les aportacions i limitacions.

3.1 Evolució cap a l'Edge Computing i Desplegament MLOps

L'expansió de l'Internet de les Coses (IoT) ha forçat una transició del processament centralitzat al núvol cap a l'*Edge Computing*, apropant el còmput a l'origen de les dades per reduir la latència i l'ús d'amplada de banda [**?**]. En aquest context, el desplegament de models d'Intel·ligència Artificial (MLOps) en dispositius perimetrals presenta reptes crítics de maquinari. Treballs recents posen de manifest que els dispositius *Edge* pateixen colls d'ampolla severs en memòria RAM i capacitat de CPU quan intenten processar matrius multidimensionals (tensors) en temps real, obligant a optimitzar tant els models com la infraestructura subjacent [**?**].

3.2 L'Impost Arquitectònic de Kubernetes (K3s, MicroK8s)

Gran part de la literatura recent aposta per adaptar l'estàndard *Cloud-Native* mitjançant distribucions reduïdes de Kubernetes com K3s o MicroK8s [**?**]. Tot i que K3s aconsegueix eliminar dependències pesades empaquetant el pla de control en un únic binari amb bases de dades lleugeres (SQLite), estudis de rendiment assenyalen que l'orquestrador manté un "Impost Arquitectònic" estructural ineludible [**?**]. Aquest sobrecost prové del cicle de reconciliació constant de l'agent `kubelet`, la gestió del `containerd` i, molt especialment, l'encapsulament de la Xarxa Definida per Programari (SDN) operada per connectors com Flannel. Recerques prèvies conclouen que, en dispositius amb limitacions severes (menys de 4 GB de RAM), aquests processos en segon pla acaparen recursos vitals que haurien de destinar-se exclusivament a la inferència computacional [**?**].

Dins de l'ecosistema d'orquestradors per a l'Edge, K3s competeix directament amb solucions com MicroK8s (desenvolupat per Canonical) i k0s (Mirantis). Diversos benchmarks independents assenyalen que, si bé MicroK8s ofereix una integració nativa excel·lent amb l'ecosistema Ubuntu, tendeix a presentar una petjada de memòria superior a causa de la seva arquitectura basada en snaps. En aquest sentit, K3s és sovint considerat l'estàndard de facto per a entorns amb restriccions severes, motiu pel qual ha estat la distribució escollida per representar el model distribuït en aquesta comparativa.

Un dels components que genera més sobrecost estructural en Kubernetes és el connector de xarxa (CNI). Solucions tradicionals com Flannel o Calico basen el seu funcionament en l'encapsulament de paquets (mitjançant túnels VXLAN o IPIP) i l'ús de regles iptables. La literatura recent apunta cap a una nova generació de connectors basats en eBPF (com Cilium), els quals eliminen gran part de l'overhead de xarxa en processar els paquets directament a l'espai del kernel. Malgrat aquesta

evolució, la configuració per defecte de K3s manté Flannel per la seva extrema simplicitat, un factor que serà avaluat críticament en aquest treball.

3.3 Execució Nativa, Contenidors Rootless i Systemd

Una línia d'investigació alternativa planteja prescindir de l'orquestració distribuïda per maximitzar la potència de càlcul mitjançant motors autònoms *Standalone* [**?**]. L'evolució cap a contenidors *rootless* (sense privilegis d'administrador) liderada per tecnologies com Podman, combinada amb la gestió de cicles de vida directament a través del *kernel* de Linux mitjançant **Systemd** (Quadlets), ha permès eliminar el sobrecost històric de dimonis centralitzats com el de Docker [**?**]. Aquest enfocament permet operar amb una xarxa de tipus pont (bridge) local, evitant l'encapsulament de paquets propi de les xarxes virtuals distribuïdes (SDN), la qual cosa redueix la latència respecte a solucions com Flannel/VXLAN. No obstant això, la literatura adverteix que el model *Standalone* sacrifica dràsticament l'escalabilitat i la tolerància a fallades, obligant els enginyers a utilitzar eines d'aprovisionament declaratiu com **Ansible** per emular un pla de control extern [**?**].

3.4 Eficiència en la Transmissió de Tensors (Protocols i Serialització)

En l'àmbit específic de la transmissió de grans volums de dades entre el pla de control i els nodes perimetrals, l'estat de l'art demostra empíricament la ineficiència dels estàndards web tradicionals. L'ús de HTTP/REST amb text pla (JSON) genera bloquejos inassumibles a causa de l'alt cost computacional de deserialització (*parsing*) [**?**].

Com a alternativa, la recerca es decanta per protocols de serialització binària com gRPC (sobre HTTP/2) o solucions directes *brokerless* com ZeroMQ [**?**]. Estudis recents focalitzats en el processament científic i MLOps destaquen que l'ús de formats binaris sense esquema (*schema-less*), com MessagePack, permet aplicar tècniques de *Zero-Copy* directament sobre matrius a la memòria RAM (`numpy`). Això accelera dràsticament l'entrada de dades a la xarxa neuronal sense ofegar el processador [**?**]. Tanmateix, la gran majoria d'aquests bancs de proves s'executen en entorns de xarxa ideals, sense analitzar com l'encapsulament de paquets d'un orquestrador subjacent (com els túnels VXLAN de Kubernetes) pot arribar a degradar aquest alt rendiment teòric.

3.5 El buit científic que omple aquest treball

Tot i l'abundant literatura sobre l'adopció d'arquitectures Cloud-Native i l'optimització de models d'Intel·ligència Artificial a l'extrem perimetral, fins on abasta la nostra revisió, no s'han identificat treballs previs que aïllin i quantifiquin empíricament l'impacte exacte de l'orquestrador sobre el rendiment brut de la xarxa combinat amb serialització binària dinàmica. Tampoc s'han identificat estudis que hagin mesurat, en aquest escenari concret, quants recursos es poden recuperar en substituir Kubernetes per una pila nativa basada en Systemd i Podman.

Aquest treball pretén contribuir a omplir aquest buit, aportant dades objectives extretes de proves d'estrès sobre el sobrecost operatiu de Kubernetes a l'Edge en un cas d'ús real de visió per computador. Els resultats mostren, amb dades pròpies, com l'alternativa nativa de Systemd redueix el consum de memòria RAM en repòs, redueix la penalització de les xarxes virtuals sobre l'amplada de banda amb ZeroMQ, i millora el Temps Mitjà de Recuperació (MTTR) respecte al model distribuït de K3s en les condicions avaluades.

3.6 Síntesi de Treballs Relacionats

La Taula **??** presenta un resum comparatiu dels treballs més rellevants analitzats en aquesta revisió, destacant-ne l'enfocament principal i la limitació que justifica la present recerca.

Referència	**Plataforma Avaluada**	**Mètrica Principal**	**Limitació respecte a aquest TFG**
Morabito et al. [**?**]	K3s vs MicroK8s	Consum de RAM i CPU	No avalua aplicacions d'IA ni l'impacte de la xarxa (CNI) sota estrès.
Casola et al. [**?**]	Contenidors Standalone	Temps de desplegament	No compara directament l'arquitectura nativa amb Kubernetes en el mateix hardware.
Zhao et al. [**?**]	ZeroMQ i Formats Binaris	Throughput de xarxa	S'executa en entorns ideals, sense mesurar l'impacte d'una xarxa virtualitzada (SDN).
Aquest treball	**Systemd vs K3s**	**RAM, MTTR i Throughput**	**Avalua el sobrecost combinat d'orquestració i xarxa en inferència MLOps real.**

Taula 3.1: Comparativa d'estudis relacionats amb l'orquestració i rendiment a l'Edge.

Capítol 4

Metodologia

Aquest capítol descriu el disseny experimental dissenyat per avaluar i quantificar l'impacte sobre el rendiment de Kubernetes (K3s) en comparació amb una arquitectura nativa basada en Systemd. L'estudi s'articula mitjançant una comparativa A/B automatitzada per garantir la reproductibilitat de totes les proves.

4.1 Especificacions de l'Entorn Experimental (Testbed)

Per contextualitzar adequadament l'avaluació en maquinari de capacitats reduïdes i assegurar la replicabilitat de l'experiment, la Taula **??** recull les característiques tècniques de la plataforma de proves virtualitzada.

Component / Paràmetre	**Especificació tècnica**
Arquitectura base (amfitrió)	Intel Core i9-14900HX (x86_64)
Entorns de virtualització	WSL2 (Node Mestre) / VMware Workstation Player 17 (Nodes *Workers*)
S.O. i nucli (Mestre)	Ubuntu 22.04.5 LTS – Linux Kernel 6.18.x (Microsoft Standard)
S.O. i nucli (*Workers*)	Ubuntu 24.04.4 LTS (mode *headless*) – Linux Kernel 6.8.0-138-generic
Recursos Node Mestre	32 vCPUs, 16 GB RAM, 1 TB d'emmagatzematge
Recursos *Workers* (Systemd - proves)	2 vCPUs, 2 GB RAM, 20 GB d'emmagatzematge NVMe/SATA
Recursos *Workers* (Definitiu)	2 vCPUs, 4 GB RAM, 35 GB d'emmagatzematge NVMe/SATA
Versions de programari	Podman (v4.9.3), Systemd (v255), K3s (v1.35.4+k3s1), Ansible (v2.17.14), Go (v1.25.9)
Xarxa i connectivitat	Gestió via *Netplan* (DHCP), interfície `ens33`, CNI Flannel (`10.42.1.0/24`)
Velocitat de l'enllaç de xarxa	1 Gbps (Adaptador virtualitzat per VMware)

Taula 4.1: Característiques generals de l'entorn experimental.

A causa de la petjada de memòria estructural associada al pla de control de Kubernetes, es van establir dues configuracions de maquinari diferenciades en funció de la fase del projecte, tal com es detalla a la Taula **??**:

- **Fase nativa preliminar (Systemd + Podman):** Durant les proves de viabilitat inicials, els nodes perimetrals es van configurar amb un perfil altament restrictiu de **2 vCPUs**, **2 GB de**

RAM i **20 GB** d'emmagatzematge. Aquest dimensionament va permetre avaluar l'operativitat de la pila nativa en condicions de recursos mínims.

- **Fase d'orquestració (K3s):** L'execució de la pila de K3s sobre l'entorn inicial de 2 GB de RAM va provocar fallades d'execució per esgotament de memòria (*Out-Of-Memory*, OOM). Per aquest motiu, va caldre redimensionar els nodes a **2 vCPUs**, **4 GB de RAM** i **35 GB** d'emmagatzematge de disc.

Perquè la comparativa de rendiment sigui justa i utilitzi exactament el mateix maquinari, **tots els tests finals d'aquest estudi es van recollir utilitzant la configuració de 4 GB de RAM i 35 GB de disc, tant per a Systemd com per a K3s.**

4.2 Disseny Iteratiu i Selecció de Protocols

L'estudi es va estructurar en quatre etapes iteratives amb l'objectiu d'identificar prèviament la configuració de xarxa més eficient i impedir que la capa de comunicació actués com a coll d'ampolla durant l'avaluació comparativa:

- **Fase 1: Exploració inicial de protocols en entorn natiu:** Es va mesurar l'impacte dels protocols de transport (HTTP, gRPC, ZeroMQ) i dels formats de codificació (text pla enfront de binari) sobre la pila base de Systemd utilitzant la càrrega de treball MNIST. L'anàlisi de paràmetres com el temps de resposta, la taxa d'èxit i el consum de recursos va confirmar que la combinació de ZeroMQ amb formats binaris oferia el rendiment més sòlid.
- **Fase 2: Avaluació de la serialització binària (Protobuf enfront de MessagePack):** Dins de les opcions binàries, es va dur a terme una comparativa a quatre bandes: HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack. Tot i les propietats de Protobuf en entorns estructurats, MessagePack va resultar més adient a causa de les característiques del conjunt MNIST:
  - *Sobrecàrrega en Protobuf:* Definir la matriu de la imatge mitjançant el camp `repeated int32` en el fitxer `.proto` incrementa la mida del missatge de 784 bytes a més de 2 KB, provocant un consum excessiu de CPU durant els cicles de codificació i descodificació píxel a píxel. Encara que recórrer al tipus `bytes` esmorteïa aquesta penalització, eliminava l'avantatge del tipat estricte propi de la solució.
  - *Eficiència en MessagePack:* Aquest mecanisme permet empaquetar directament el flux binari sense processament intermedi (*bin type*) a partir de les estructures de memòria, minimitzant l'ús de processador i mantenint una transmissió compacta.
- **Fase 3: Validació del protocol sobre l'orquestrador:** El comportament observat amb ZeroMQ i MessagePack es va reproduir sobre el clúster de Kubernetes, verificant que el patró d'eficiència es mantenia constant independentment de la plataforma subjacent. Com a distribució distribuïda, es va optar per K3s pel seu disseny orientat específicament a plataformes *Edge*.
- **Fase 4: Comparativa arquitectònica seleccionada:** Amb el canal de comunicació fixat en ZeroMQ-MessagePack per descartar limitacions de xarxa, es van executar les proves d'estrès de referència per contraposar directament l'arquitectura nativa (Systemd) amb l'entorn distribuït (K3s).

4.3 Selecció del Model d'IA

Perquè la càrrega fos representativa d'un cas d'ús real de visió artificial, es va descartar l'ús d'un Perceptró Multicapa (MLP) simple en favor d'una Xarxa Neuronal Convolucional (CNN). Aquest

tipus de xarxa aprofita la correlació espacial dels píxels mitjançant operacions matricials intensives en capes convolucionals i denses. Els tensors generats sotmeten el processador i la memòria a un volum d'inferència continu, permetent avaluar el rendiment d'ambdues solucions sota una càrrega computacional representativa.

4.4 Topologia i Estratègia de Desplegament (Patró *Air-Gapped*)

L'arquitectura de la prova separa el node de gestió dels nodes d'execució (*Workers*). Amb la intenció d'aïllar els resultats de variacions derivades de la connectivitat externa o del trànsit d'Internet, es va seguir una metodologia *Air-Gapped*:

- **Construcció centralitzada:** La generació i compilació dels binaris i imatges es centralitza al node mestre per no penalitzar la capacitat de càlcul dels nodes perimetrals.
- **Distribució local:** Els artefactes resultants es transfereixen directament a l'emmagatzematge intern de cada *worker* abans d'iniciar les mesures.
- **Separació de responsabilitats:** Tant en Systemd com en K3s, l'execució de les tasques d'inferència queda confinada als nodes perimetrals, assegurant que el pla de gestió no interfereixi en els mesuraments de rendiment.

4.5 Automatització i Reproduïbilitat (*Pipeline* d'execució)

La recollida de dades s'ha articulat mitjançant un flux de treball completament automatitzat per evitar desviacions manuals. Cada iteració segueix la seqüència següent:

- **Purga i desplegament inicial:** S'esborren els contenidors i volums previs i s'elimina la memòria cau per reiniciar l'entorn des d'un estat base homogeni abans de cada cicle.
- **Període d'estabilització:** S'apliquen intervals d'espera programats per evitar fenòmens d'estrangulament tèrmic (*thermal throttling*) al maquinari i permetre que els serveis base assoleixin un estat estacionari abans de mesurar els consums.
- **Cicle de càrrega:** S'aplica un patró de sol·licituds concurrents continuat per avaluar el comportament del sistema sota condicions de pressió operativa sostinguda.
- **Centralització de registres:** Les lectures generades s'extreuen dels registres del sistema operatiu i s'emmagatzemen automàticament en un fitxer estructurat de tipus CSV per facilitar el tractament estadístic posterior.

4.6 Tècniques de Mesura i Mètriques d'Avaluació

L'impacte de cada arquitectura es quantifica a través de quatre indicadors principals d'infraestructura i disponibilitat:

- **Temps de Desplegament (T_{deploy}):** Interval temporal absolut requerit per aprovisionar el node. Inclou des de l'inici de l'execució del *playbook* d'Ansible fins que els contenidors (o *Pods*) assoleixen l'estat operatiu.
- **Rendiment Global (*Throughput*):** Avalua la capacitat de la xarxa i el processador sota estrès. Es defineix com la quantitat d'imatges (tensors) processades amb èxit per unitat de temps. Si N és el nombre total d'imatges enviades i T_{total} és l'interval de temps en segons des de la primera petició fins a l'última resposta:

$$Throughput = \frac{N}{T_{total}} \quad [\text{img/s}]$$

- **Consum Mitjà de Recursos (RAM i CPU):** El consum base (en repòs) s'avalua capturant k mostres discretes del monitoratge del sistema (mitjançant `ps` o cgroups) durant un període d'estabilització. La petjada de memòria es calcula com la mitjana aritmètica d'aquestes lectures:

$$\overline{RAM}_{repòs} = \frac{1}{k} \sum_{i=1}^{k} RAM_i \quad [\text{MB}]$$

- **Temps Mitjà de Recuperació (MTTR):** Temps requerit pel sistema (en mil·lisegons) per restablir el servei després de provocar la terminació forçada del procés principal mitjançant senyals de caiguda abrupta.

4.6.1 Tractament de Valors Atípics (Outliers)

Durant els assajos de tolerància a fallades sobre Kubernetes, es va registrar un valor anòmal en una de les iteracions, amb un temps de recuperació de 261.450 ms (~4,3 minuts). Aquest retard va ser causat per una retenció temporal (*lock*) en l'alliberament de la xarxa virtual gestionada per Flannel.

En comptes de suprimir la dada o recórrer a la imputació estadística, s'ha optat per segregar aquest registre de l'anàlisi de tendència central. Així, les mètriques resum (mitjana, mediana i desviació estàndard) es presenten excloent aquest valor per caracteritzar el comportament nominal de l'orquestrador en condicions d'estabilitat. No obstant això, el cas s'analitza de manera monogràfica en el capítol de resultats, ja que posa de manifest una vulnerabilitat pròpia dels mecanismes de coordinació en xarxes virtuals definides per programari (SDN).

Capítol 5

Implementació

Aquest capítol detalla la implementació tècnica de l'entorn de proves, descrivint l'arquitectura de programari, les eines de desenvolupament utilitzades i els procediments d'automatització dissenyats per extreure dades comparatives fiables.

5.1 Tecnologies, Llenguatges i Eines Utilitzades

- **Ansible:** Eina central per orquestrar la preparació de la infraestructura en ambdós entorns. Permet la injecció dinàmica de variables (com `experimento_path`) per avaluar múltiples protocols de xarxa de manera desatesa.
- **Podman i Systemd:** Tecnologies base per a l'arquitectura nativa. Podman construeix les imatges ajustant el gestor de recursos (`--cgroup-manager=cgroupfs`) i les executa en mode *rootless*. El cicle de vida es delega a fitxers declaratius Quadlet gestionats per Systemd.
- **K3s i `kubectl`:** Distribució lleugera de Kubernetes utilitzada com a referència d'orquestrador perimetral.
- **Python (PyTorch i Lògica de Xarxa):** Llenguatge principal de programació del projecte. S'ha utilitzat per desenvolupar tota la lògica dels nodes mestre i treballador (*Master/Worker*), i per implementar els diferents protocols de comunicació avaluats (HTTP-JSON, gRPC, ZeroMQ). A més, s'ha utilitzat programació en Python per automatitzar la descàrrega del conjunt de dades MNIST del núvol i per dissenyar, entrenar i exportar el model d'Intel·ligència Artificial mitjançant la llibreria especialitzada PyTorch.
- **Bash:** Llenguatge utilitzat per construir els *scripts* d'automatització que governen l'estrès de la infraestructura, l'enginyeria del caos i la invocació repetitiva de les proves de xarxa.

5.2 Disseny i Entrenament del Model d'IA (PyTorch)

Per garantir que la prova d'estrès generés una càrrega de processament d'imatges realista abans de l'avaluació de la infraestructura, es va dissenyar i entrenar un model d'Intel·ligència Artificial personalitzat amb el *dataset* MNIST (80% per a entrenament, 20% per a validació), descartant l'ús d'un Perceptró Multicapa (MLP) bàsic en favor d'una Xarxa Neuronal Convolucional (CNN).

- **Justificació de l'Arquitectura:** La decisió d'optar per una CNN respon a la seva superioritat en tasques de visió per computador, ja que permet aprofitar la correlació espacial dels píxels de les imatges.
- **Bloc d'Extracció de Característiques:** Està compost per dues capes convolucionals (`Conv2d`) que extreuen 32 i 64 mapes de característiques, respectivament. Cada convolució va seguida

d'una funció d'activació no lineal (`ReLU`) i d'una capa d'agrupació (`MaxPool2d`) per reduir la dimensionalitat i el cost computacional.

Listing 5.1: Crear un model de Xarxa Neuronal Convolucional (CNN) amb PyTorch.

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.reshape(-1, 64 * 7 * 7)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Initialize the CNN model
model = CNN()
print(model)
```

- **Bloc de Classificació:** Un cop aplanats els mapes de característiques mitjançant l'operació `reshape`, la informació es transmet a través de dues capes denses (*Fully Connected* o `Linear`). Aquestes capes redueixen progressivament les dimensions fins a produir una sortida final de 10 nodes, que representen les probabilitats d'encert per als 10 dígits possibles (del 0 al 9).
- **Paràmetres d'Entrenament (MLOps):** L'optimització dels pesos de la xarxa neuronal s'ha dut a terme aplicant l'algoritme de Descens de Gradient Estocàstic (SGD), configurat amb una taxa d'aprenentatge de 0.01 i un impuls (*momentum*) de 0.9. La funció de cost seleccionada ha estat l'Entropia Creuada (`CrossEntropyLoss`), i s'ha establert un únic cicle d'entrenament (1 època) processant les imatges en blocs (*batch size*) de 64 unitats.

Justificació de l'Entrenament d'1 Època:

Per a l'avaluació del sistema, l'entrenament de la CNN es va configurar deliberadament per executar-se durant una única època (1 Epoch). Aquesta decisió arquitectònica es fonamenta en tres criteris clau:

1. **Abast del Projecte:** L'objectiu principal d'aquest estudi és la comparativa de Systemd amb Kubernetes a l'entorn *Edge*, no l'optimització d'hiperparàmetres en models de *Deep Learning*.
2. **Evidència de Convergència:** Com s'observa a la corba d'aprenentatge, l'error d'entrenament (*Train Loss*) experimenta una caiguda dràstica i s'estabilitza abans de finalitzar la primera iteració completa del *dataset*.
3. **Eficiència Computacional:** En finalitzar aquesta única època, el model assoleix una precisió de validació (*Validation Accuracy*) propera al 98%. Aquest nivell d'encert és prova empírica suficient per certificar que el *pipeline* d'inferència i la transmissió de tensors funcionen correctament d'extrem a extrem, evitant un consum innecessari de temps i recursos al node Orquestrador.

5.3 Infraestructura com a Codi: Arquitectura Nativa (Systemd)

Aquesta secció detalla la Infraestructura com a Codi (IaC) basada en Ansible per a l'aprovisionament automatitzat i la gestió del cicle de vida dels nodes (Mestre i *Workers*) en l'entorn de processament perimetral natiu.

Arquitectura i Implementació Tècnica

El sistema està dissenyat sota el paradigma de contenidors *rootless* per maximitzar la seguretat perimetral, delegant el control dels serveis directament a Systemd:

- **Systemd Linger:** S'habilita la persistència de processos en segon pla per a l'usuari del sistema (`littledragon`) executant `loginctl enable-linger`. Això garanteix que els serveis es mantinguin actius un cop tancada la sessió SSH.
- **Podman Quadlets:** S'elimina l'ús tradicional de `docker-compose`. L'estat desitjat es declara mitjançant Quadlets, injectant plantilles Jinja2 directament a `~/.config/containers/systemd/` per generar xarxes (`red.network`) i contenidors (`worker.container`) de forma nativa.
- **Aïllament i Resiliència Local:** El sistema crea una xarxa virtual dedicada (`red-interna`). A més, la unitat de Systemd aplica polítiques d'auto-recuperació (`Restart=always` amb reinici immediat) per garantir la resiliència del *Worker* davant de fallades.

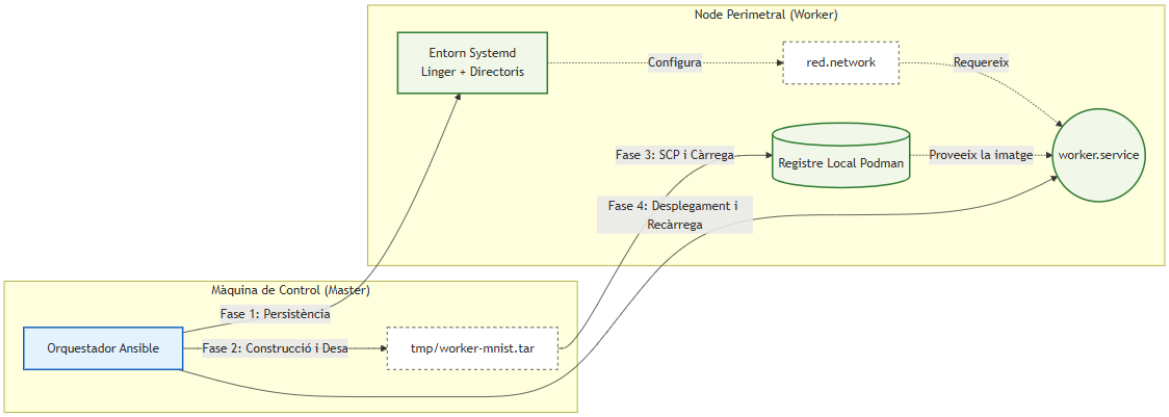


Figura 5.1: Arquitectura de l'entorn nativa amb Systemd.

Disseny Modular i Desplegament Dinàmic (*Air-Gapped*)

El *pipeline* de desplegament és completament independent del protocol de comunicació. La injecció de l'entorn es controla dinàmicament en temps d'execució sobrescrivint la ruta de l'experiment amb variables addicionals (`-e "experimento_path=..."`), permetent pivotar l'arquitectura sense alterar el codi font. S'utilitza un patró *Air-Gapped*:

- Ansible itera sobre el protocol seleccionat i compila la imatge OCI centralitzadament al node de control.
- La imatge s'empaqueta en un artefacte immutable (`.tar`) i es transfereix via SSH.
- Es carrega directament al motor local (`podman load`), alliberant l'*Edge* de la descàrrega de dependències.

Seguretat i Aprovisionament Desatès (*Zero-Touch Provisioning*)

Per assolir una automatització absoluta sense intervenció humana i protegir el repositori de codi, l'escalada de privilegis (`sudo`) s'injecta des del fitxer local `group_vars/workers.yml` (exclòs via

`.gitignore`), eliminant la necessitat d'introduir contrasenyes interactives (`-K`) durant el desplegament.

Documentació d'Execució i Benchmarking

El procés està governat pels següents components d'automatització:

- **`playbook.yml`**: Activa el *Linger*, desplega la xarxa, distribueix l'artefacte `.tar`, instància el Quadlet i habilita el `worker.service`.
- **`clean.yml`**: Executa una purga agressiva aturant els serveis i forçant l'esborrat de xarxes internes per evitar interferències entre proves.
- **`generate_benchmarks.sh`**: Orquestra el cicle d'avaluació executant el desplegament, capturant el consum base de RAM/CPU, aplicant Enginyeria del Caos (mitjançant `podman kill` o `kubectl delete pod`) per mesurar el MTTR i, finalment, llançant la simulació d'inferència per consolidar les mètriques en un *Data Lake* CSV.

5.4 Infraestructura com a Codi: Arquitectura Distribuïda (K3s)

Aquesta secció detalla l'adaptació de l'entorn cap a un model declaratiu i centralitzat utilitzant K3s, una distribució lleugera de Kubernetes optimitzada per a l'*Edge* (IoT/ARM) [**?**].

Arquitectura i Implementació Tècnica

- **Motor de Contenidors (`containerd`):** Se substitueix Podman pel motor intern de Kubernetes. La gestió d'imatges queda sota el control absolut del component `Kubelet` de cada node.
- **Manifestos Declaratius (YAML):** S'elimina l'ús de Quadlets. L'estat es defineix al directori `k8s-manifests/`, aplicant la configuració al clúster via `kubectl apply`.
- **Xarxa Definida per Programari (SDN):** L'aïllament i la connectivitat depenen de la xarxa virtual del clúster (CNI Flannel), permetent la comunicació transparent entre nodes.
- **Resiliència i Auto-Recuperació:** El bucle de reconciliació (*Reconciliation Loop*) de Kubernetes garanteix que l'estat real coincideixi amb el declarat, aixecant nous *Pods* automàticament davant de caigudes.

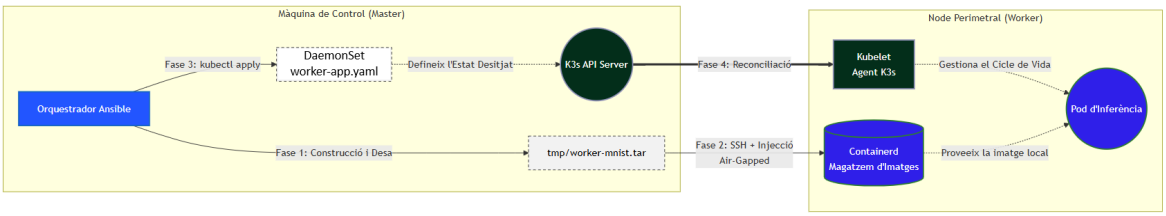


Figura 5.2: Arquitectura de l'entorn nativa amb Kubernetes.

Disseny Modular i Desplegament Dinàmic (*Air-Gapped*)

L'arquitectura manté l'enfocament modular i dinàmic (`-e "experimento_path=..."`) sota el patró *Air-Gapped* adaptat a K3s:

- Compilació de la imatge OCI al node mestre.
- Empaquetament en un artefacte `.tar` i transferència via SSH.
- Injecció directa al *namespace* intern de K3s utilitzant `k3s ctr images import`, evitant dependre de registres externs.

Seguretat i Inventari

Igual que a l'entorn natiu, la topologia es defineix a l'`inventory.ini` separant el grup `[master]` (execució local per optimitzar la compilació) del grup `[workers]`. La seguretat es gestiona mitjançant credencials aïllades a `group_vars/workers.yml` per facilitar un *Zero-Touch Provisioning*. Els serveis s'exposen a través d'un *Service* tipus *NodePort* pel port 8000.

Documentació d'Execució i Benchmarking

- **`playbook.yml`**: S'encarrega de la transferència *Air-Gapped*, injecta els manifestos YAML al clúster i delega al `Kubelet` la inicialització fins a assolir l'estat *Ready*.
- **`clean.yml`**: Executa una purga lògica eliminant els recursos descrits als manifestos i neteja la memòria cau d'imatges amb `k3s crictl rmi` per evitar la contaminació de mètriques (processos *zombie*).
- **`generate_benchmarks.sh`**: Mapeja dinàmicament el nom lògic del node en K3s a la seva IP física per capturar el consum global pur. Força l'eliminació abrupta d'un *Pod* (`kubectl delete pod --force`) per mesurar el MTTR monitoritzant el bucle de reconciliació, i executa la prova d'estrès bolcant les dades mitjançant Regex al CSV consolidat.

5.5 Disseny del Protocol de Comunicació i Inferència d'IA

Un cop desplegada la infraestructura, la transmissió de les dades s'efectua mitjançant una arquitectura de missatgeria síncrona *Request-Reply* (REQ-REP) basada en TCP pur amb ZeroMQ. Després d'avaluar diverses alternatives de serialització binària, es va descartar Protobuf en favor de MessagePack per optimitzar l'agilitat operativa en l'ecosistema d'aprenentatge automàtic.

Aquesta implementació elimina la necessitat de precompilar contractes estrictes (`.proto`), adoptant un format *schema-less* (sense esquema) que s'integra nativament amb els diccionaris de Python i distribueix la càrrega entre el Pla de Control i el Pla de Dades, tal com s'il·lustra a les figures globals de l'arquitectura:

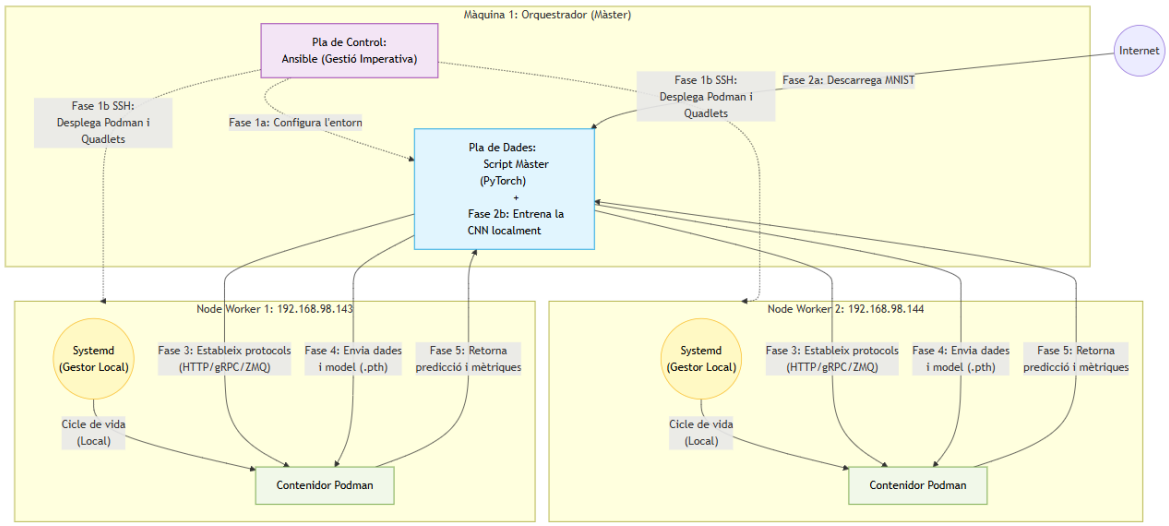


Figura 5.3: Topologia completa de la inferència d'IA sobre Systemd.

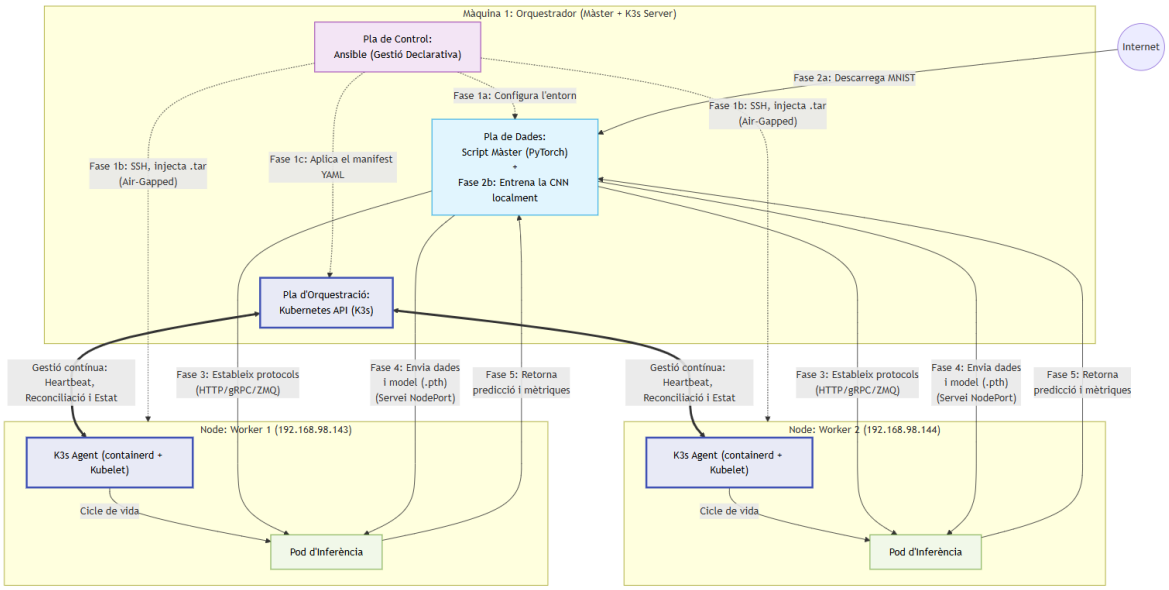


Figura 5.4: Topologia completa de la inferència d'IA sobre K3s.

El flux d'execució i l'enrutament s'estructuren de la següent manera:

- **Orquestració i Estructuració Dinàmica (Node Mestre):** El mestre assumeix el control total de l'entrenament (*Offline*) i la injecció de dades. En lloc d'instanciar classes precompilades, empaqueta els bytes del model (`.pth`) i els tensors d'imatges directament en un diccionari estàndard (`{'model_data': model_bytes}`). Aquesta estructura es comprimeix dinàmicament a format binari utilitzant `msgpack.packb(..., use_bin_type=True)` i s'injecta a la xarxa mitjançant missatgeria multiparte.
- **Gestió de Memòria i Seguretat (Nodes *Workers*):** Atès que les llibreries de MessagePack imposen límits estrictes de memòria per prevenir atacs per esgotament (OOM), el servidor perimetral sobrescriu explícitament aquests límits (`max_bin_len=200*1024*1024`) per permetre la deserialització de tensors pesats propis de PyTorch.
- **Inferència i Còpia Zero (*Zero-Copy*):** En rebre la petició d'inferència, el *Worker* recupera el tensor mitjançant la clau del diccionari i aplica tècniques de còpia zero (`np.frombuffer`). Això permet assignar el bloc de memòria directament a la Xarxa Neuronal Convolucional (CNN), invertint tot l'esforç de la CPU en el càlcul matemàtic (`torch.no_grad()`) sense duplicar les dades a la RAM.
- **Enrutament segons l'Arquitectura:** En l'entorn natiu (Systemd), el diccionari MessagePack s'envia directament al port local del contenidor. En l'entorn distribuït (K3s), el mestre injecta els paquets a través d'un servei *NodePort* exposat al port 30000, delegant a l'orquestrador l'enrutament intern cap al *socket* del *Pod* corresponent.

5.6 Automatització del *Pipeline* i *Data Lake*

L'avaluació es fonamenta en una jerarquia de tres *scripts* de Bash que actuen de manera ininterrompuda per evitar qualsevol biaix manual. En el nivell superior, un **orquestrador mestre híbrid** governa el cicle de vida global de l'experiment. La seva funció principal és gestionar la transició entre plataformes i garantir la integritat de les condicions inicials, forçant **reinicis de les màquines virtuals** (`sudo reboot`) als nodes perimetrals i aplicant pauses de refredament de 120 segons per

aconseguir un **entorn experimental estabilitzat** i evitar la degradació del rendiment per estrangulament tèrmic (*Thermal Throttling*).

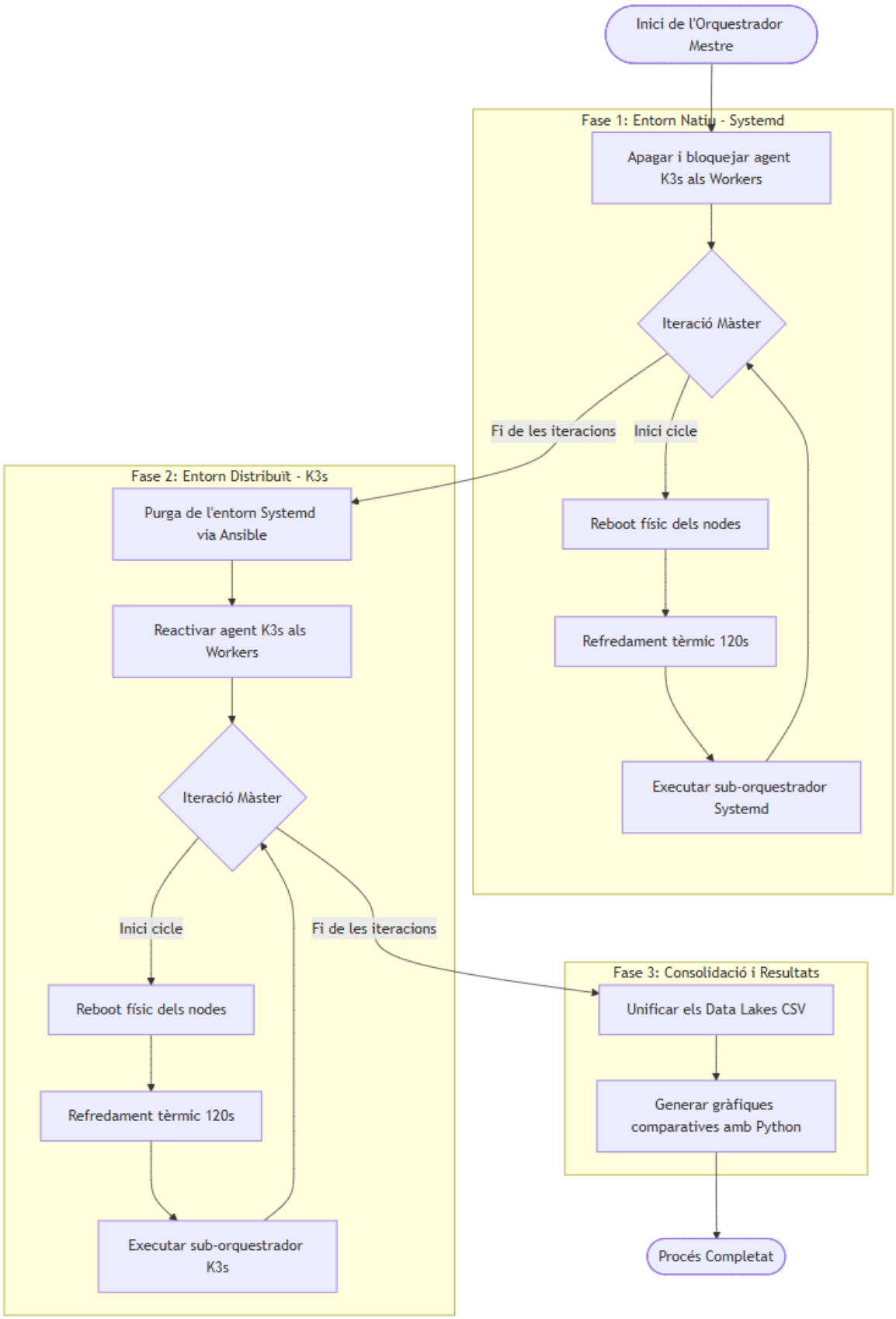


Figura 5.5: Flux del pipeline d'execució automatitzada.

Un cop l'entorn experimental està estabilitzat, aquest orquestrador mestre invoca els *scripts* específics d'avaluació (un dissenyat exclusivament per a Systemd i un altre per a K3s). Aquests suborquestradors iteren seqüencialment sobre els protocols executant les següents fases:

- **Inicialització i Estabilització:** S'executa una neteja total de l'entorn previ i es desplega la infraestructura des de zero. S'apliquen pauses d'estabilització (de 15 segons en K3s) per permetre que els processos en segon pla com el *metrics-server* s'assentin i recullin dades fiables del consum base.
- **Enginyeria del Caos:** Destrucció abrupta del procés actiu i mesura mil·limètrica del Temps Mitjà de Recuperació (MTTR) mitjançant consultes iteratives a l'estat del sistema.
- **Bucle d'Estrès i Extracció (*Data Lake*):** Execució de 10 llançaments consecutius del client Python per saturar la xarxa. Un cop finalitzats, un *script* de Python incrustat dins de Bash analitza els registres en text pla mitjançant expressions regulars (Regex) i bolca dinàmicament totes les mètriques estructurades (latència, *Throughput*, memòria màxima, etc.) al fitxer consolidat `resultados_globales.csv`.

Listing 5.2: Extracció de mètriques mitjançant expressions regulars cap al Data Lake CSV

```
# Extracció d'infraestructura
t_deploy = get_avg(log_file, r"T_deploy total:\s*([0-9\.]+)")
cpu_reposo = get_avg(log_file, r"CPU en Repòs \(Global\):\s*([0-9\.]+)")
ram_reposo = get_avg(log_file, r"RAM en Repòs \(Global\):\s*([0-9\.]+)")
mttr = get_avg(log_file, r"Temps Real Arrencada:\s*([0-9\.]+)")

# Extracció de xarxa (Inclou la taxa d'èxit)
t_total = get_avg(log_file, r"Temps Total \(s\):\s*([0-9\.]+)")
throughput = get_avg(log_file, r"Throughput \(img/s\):\s*([0-9\.]+)")
# Expressió regular tolerant: Captura "Taxa Èxit" o "Taxa d'Èxit"
tasa_exito = get_avg(log_file, r"Taxa(?: d')?\s*Èxit \(%\):\s*([0-9\.]+)")
rtt = get_avg(log_file, r"Latència RTT Mitjana \(ms\):\s*([0-9\.]+)")
t_proc = get_avg(log_file, r"Temps T_proc Mitjana \(ms\):\s*([0-9\.]+)")
ram_max = get_avg(log_file, r"Pic Màx.\. RAM Worker \(MB\):\s*([0-9\.]+)")
cpu_max = get_avg(log_file, r"CPU Mitjana \(%\):\s*([0-9\.]+)")
payload_nodo = get_avg(log_file, r"Payload Mitjana per Node \(MB\):\s*([0-9\.]+)")

# Escriure al fitxer CSV
file_exists = os.path.isfile(csv_file)
with open(csv_file, 'a', encoding='utf-8') as f:
    if not file_exists:
        f.write("Fecha, Protocolo, T_deploy, CPU_Reposo, RAM_Reposo, MTTR, T_Total, Throughput
            , Exito_%, RTT, T_Proc, RAM_Max, CPU_Max, Payload_MB\n")
    f.write(f"{fecha}, {protocol_name}, {t_deploy:.2f}, {cpu_reposo:.2f}, {ram_reposo:.2f}, {
        mttr:.2f}, {t_total:.2f}, {throughput:.2f}, {tasa_exito:.2f}, {rtt:.2f}, {t_proc:.2f
        }, {ram_max:.2f}, {cpu_max:.2f}, {payload_nodo:.2f}\n")
```

5.7 Decisions Tècniques i Resolució de Problemes

La mesura de precisió de l'MTTR va requerir desenvolupar solucions específiques per a cada tecnologia. En l'enfocament natiu, la fallada se simula amb `podman kill` i la recuperació es detecta amb bucles `while` de 0,05 segons interrogant l'estat local de `systemctl`. En contrast, a l'entorn distribuït s'ha de forçar l'eliminació del recurs a l'API (`kubectl delete pod --force --grace-period=0`) i esperar que el controlador (*DaemonSet*) instanciï el nou *Pod*, monitoritzant la transició fins a l'estat *Ready*.

5.7.1 Resolució de condicions de carrera (*Race Conditions*) en la captura de mètriques

Durant les primeres fases d'experimentació en el clúster de K3s, es va detectar una anomalia en la mesura del temps mitjà de recuperació (MTTR). A causa de l'ús de l'ordre de destrucció immediata `kubectl delete pod --force --grace-period=0`, la base de dades del clúster (API) eliminava el registre del *Pod* de forma instantània. Això provocava una condició de carrera (*race condition*) entre la mort física del contenidor antic en el node i la instanciació del nou *Pod* per part del controlador (*DaemonSet*). Com a conseqüència, l'*script* automatitzat original capturava el diagnòstic de salut (*Ready*) del *Pod* moribund abans que aquest desaparegués realment del node, registrant temps de recuperació falsament curts de pocs mil·lisegons.

Per solucionar aquesta deficiència i garantir la validesa de les mètriques, es va redissenyar l'arquitectura de l'*script* de control implementant una estratègia basada en la infraestructura fixa:

- **Iteració sobre Nodes Físics:** En lloc de fer el seguiment de *Pods* amb identificadors temporals i efímers (els quals queden descatalogats després de la fallada), el bucle de monitoratge itera de forma estàtica sobre els nodes físics de l'entorn (`node-a` i `node-b`).
- **Validació Dinàmica d'Identificadors:** L'*script* realitza consultes en temps real a l'API de K3s per extreure el nom del *Pod* allotjat en el node objectiu. El cronòmetre de recuperació no s'atura fins que es compleix una doble condició estricta: que l'estat del *Pod* sigui operatiu (*Ready*) i que el seu identificador únic hagi canviat completament respecte al *Pod* en execució abans de la prova de caos (`$NEW_POD != $POD_NAME`).

Aquest canvi de disseny metodològic elimina qualsevol interferència provocada per l'asincronia del clúster, assegurant que els temps reflectits en els resultats d'aquest treball corresponen al cicle complet i real de provisió de la nova unitat d'execució.

5.7.2 Tractament Metodològic de Fallades Estructurals (Flannel)

Durant la fase d'enginyeria del caos en l'entorn amb Kubernetes, es va documentar una incidència crítica en la infraestructura. Concretament, en una de les execucions es va produir un valor atípic (*outlier*) de 261 segons (261.450 ms) causat per un bloqueig (*lock*) temporal en l'alliberament de la IP virtual per part del connector de xarxa Flannel.

D'acord amb les indicacions metodològiques, per facilitar la comparació del comportament habitual entre ambdós sistemes, a la secció de resultats es presentaran les mètriques estadístiques de referència (mitjana, mediana, desviació estàndard, mínim i màxim) basades exclusivament en les execucions normals, ometent aquesta dada extrema. Tanmateix, aquest esdeveniment es conserva al registre i s'analitza a fons, ja que no es tracta d'un error de mesura, sinó que evidencia un mode de fallada real i crític del sistema, propi de la complexitat estructural de les xarxes definides per programari (SDN) en entorns distribuïts.

Capítol 6

Resultats i Avaluació

Aquest capítol presenta i interpreta els resultats empírics obtinguts en enfrontar l'arquitectura nativa (Systemd) contra l'orquestrador distribuït (Kubernetes/K3s), responent directament als objectius de la investigació. L'avaluació es divideix en tres fases seqüencials per assegurar el rigor tècnic: la selecció del protocol de comunicació òptim, la validació del model d'Intel·ligència Artificial, i la comparativa final d'infraestructures.

6.1 Selecció del Protocol de Comunicació

Abans d'avaluar l'impacte arquitectònic dels orquestradors, és imperatiu determinar quin és el protocol de xarxa òptim per a la transmissió massiva de tensors matemàtics. L'objectiu d'aquesta fase és aïllar la capa de transport per garantir que la serialització de les dades no actuï com a coll d'ampolla. Totes les proves d'aquesta secció s'han executat exclusivament sobre l'entorn natiu (Systemd) per registrar el límit físic del maquinari sense interferències de xarxes virtuals.

La Taula **??** presenta les mètriques consolidades, extretes de la mitjana històrica del *Data Lake* (10 execucions) després de sotmetre els nodes a càrrega extrema.

Protocol (Systemd)	T. Total (s)	Throughput (img/s)	Taxa d'Èxit	RTT (ms)	T. proc Worker (ms)	RAM Màx (MB)	Payload (MB)
HTTP/JSON	0,82	2.451,78	100 %	429,27	415,03	328,96	15,59
gRPC/Protobuf	0,23	8.594,21	100 %	223,89	200,66	309,52	2,99
ZeroMQ/Protobuf	0,22	9.092,66	100 %	216,63	202,14	277,22	2,99
ZeroMQ/MessagePack	0,22	9.053,76	100 %	215,61	199,55	284,26	2,99

Taula 6.1: Resultats consolidats de rendiment de xarxa per protocol a l'Edge (mitjana de 10 execucions).

De l'anàlisi d'aquestes mètriques i la seva representació visual, s'extreuen les següents conclusions:

1. El Coll d'Ampolla del Text Pla (HTTP/JSON) L'estàndard web tradicional es demostra totalment inviable per al processament perimetral. En no suportar transmissió binària nativa, els tensors es converteixen a text pla, inflant el *payload* fins a 15,59 MB per petició. Com s'observa a la Figura **??**, el seu rendiment queda severament estancat (~2.452 img/s). A més, la mètrica del T_{proc} (415 ms) evidencia que la CPU del *Worker* s'ofega intentant parsejar documents de text gegantins en lloc de calcular prediccions d'IA.

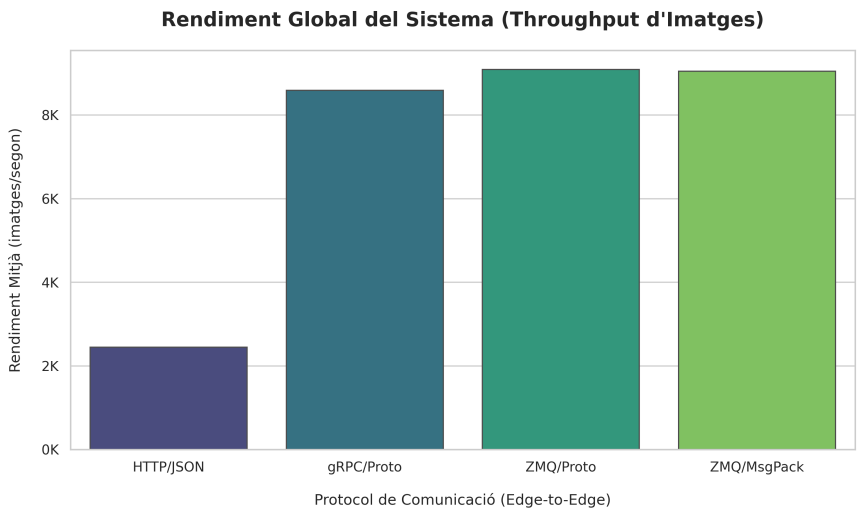


Figura 6.1: Comparativa de rendiment brut (*Throughput*) processant tensors.

2. El Salt a la Serialització Binària (gRPC) La transició cap a gRPC i Protobuf resol els problemes crítics del *baseline*. La càrrega de xarxa es redueix al seu pes matemàtic pur (2,99 MB). Aquesta eficiència permet utilitzar tècniques de *Zero-Copy* (`np.frombuffer`) a la memòria cau, reduint el temps de processament local a menys de la meitat (201 ms) i disparant el *throughput* per sobre de les 8.500 img/s.

3. Eficiència de Memòria i el Límit del Transport (ZeroMQ) Tot i l'alta velocitat de gRPC, el seu motor intern basat en HTTP/2 afegeix un *overhead* de capçaleres innecessari. Substituir-lo per connexions TCP crues (*Brokerless*) amb ZeroMQ assoleix el límit físic de velocitat. Tal com s'il·lustra a la Figura **??**, les arquitectures basades en ZeroMQ presenten la petjada de memòria (*RAM Footprint*) més baixa sota estrès (~278-283 MB), aconseguint un estalvi crític per a dispositius *Edge* limitats en comparació amb l'ecosistema gRPC.

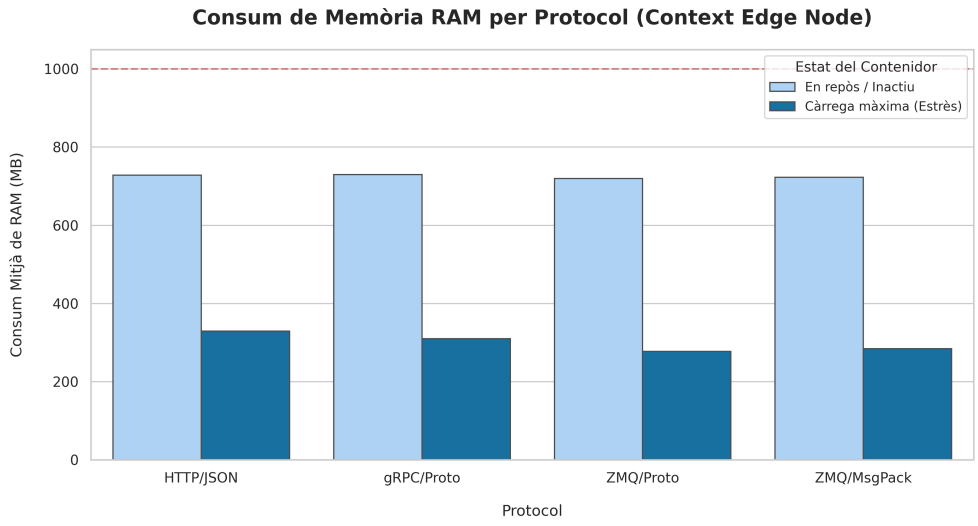


Figura 6.2: Impacte de la serialització en el consum màxim de memòria RAM als *Workers*.

4. Matriu de Trade-off La decisió arquitectònica s'il·lustra a la matriu de compensació (*Trade-off*, Figura **??**), que relaciona la capacitat de processament global amb la latència estructural. En analitzar el gràfic, s'observa que el quadrant superior esquerre (alta velocitat i mínima latència) correspon a

les configuracions que utilitzen ZeroMQ.

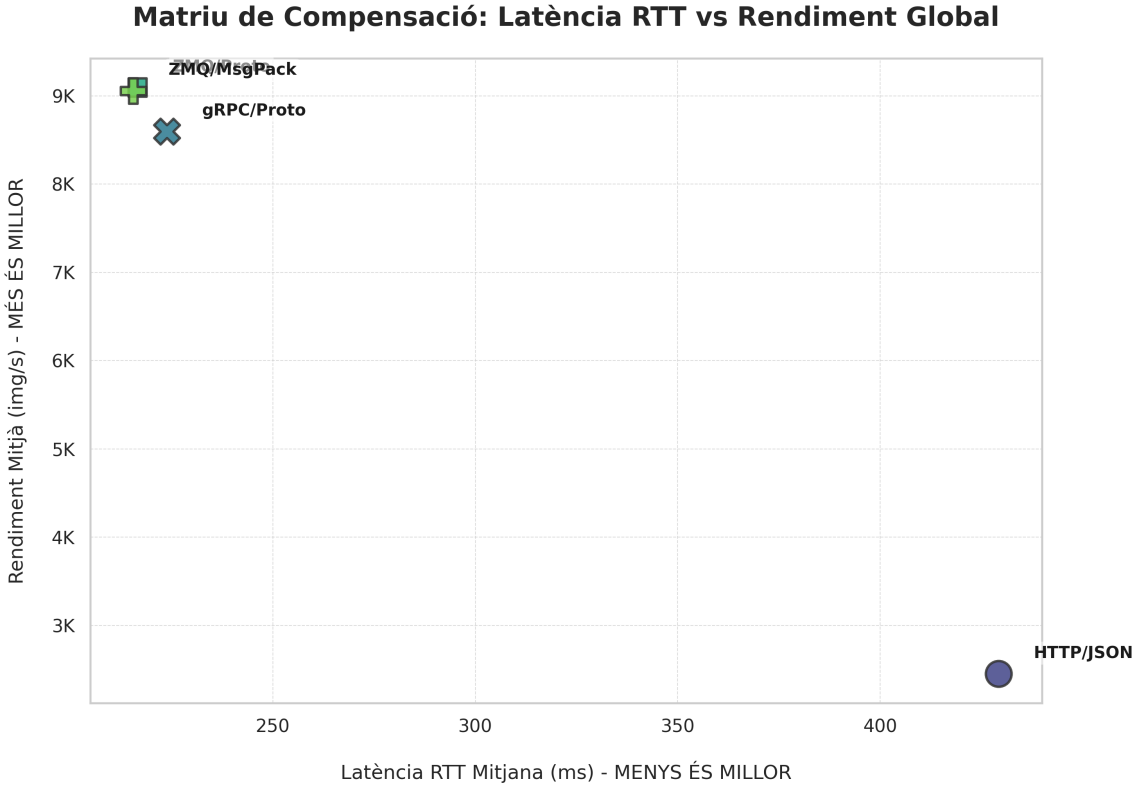


Figura 6.3: Relació de compromís (*Trade-off*) entre rendiment global i Latència RTT.

Dins d'aquest grup d'alt rendiment, **ZeroMQ combinat amb MessagePack** es consolida com l'alternativa més favorable segons els paràmetres d'estudi. Tot i presentar un rendiment marginalment inferior a Protobuf (9.054 vers 9.093 img/s), MessagePack aporta un format *schema-less* que s'integra nativament amb estructures de Python. Això elimina la complexitat de precompilar contractes estrictes (`.proto`), estabilitzant el pic de memòria cau i maximitzant l'agilitat operativa per a fluxos MLOps. Aquesta combinació s'estableix com el protocol de transport de referència per avaluar les arquitectures a la resta de l'estudi.

6.2 Rendiment del Model d'IA (CNN)

Per garantir que la infraestructura se sotmet a una càrrega d'inferència realista abans de comparar els orquestradors, és imprescindible verificar la viabilitat operativa i la integritat de les dades en el sistema distribuït. S'ha fet l'entrenament del model utilitzant el protocol triat (ZeroMQ amb MessagePack), demostrant que la partició del *dataset* i la transmissió de tensors es realitza sense corrompre el *payload*.

L'anàlisi de les mètriques visuals extretes durant la fase d'entrenament (*Offline*) confirma el comportament esperat de la Xarxa Neuronal Convolucional (CNN):

- **Convergència de l'Error (Corba de Pèrdua):**

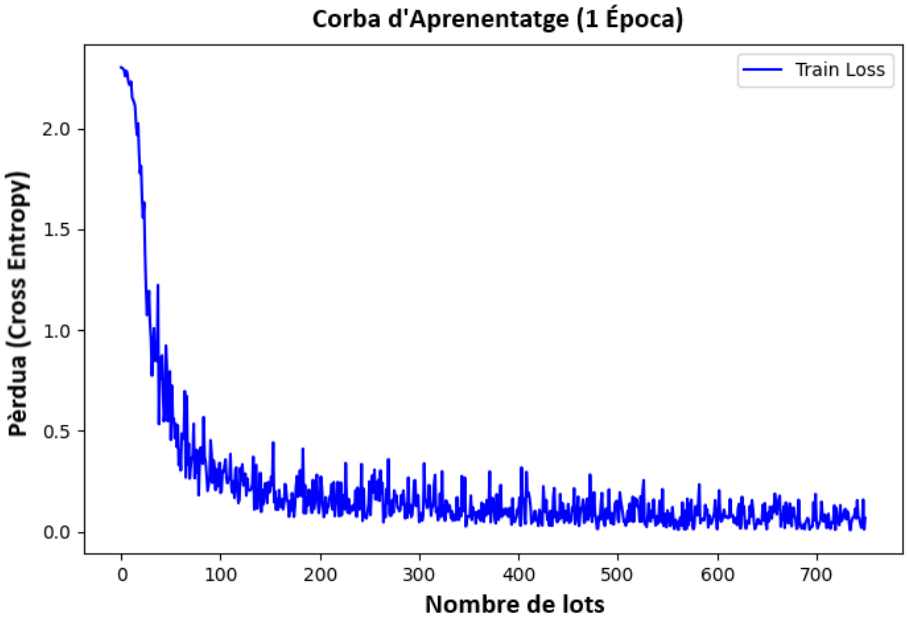


Figura 6.4: Avaluació del model CNN: Corba de pèrdua (*Train Loss*).

Com s'observa a la Figura **??**, l'error d'entrenament experimenta una caiguda dràstica i s'estabilitza abans de finalitzar la primera iteració completa del *dataset*. Això justifica empíricament la decisió d'entrenar durant una única època (1 Epoch), evitant un consum innecessari de temps al node Mestre.

- **Precisió (Matriu de Confusió):**

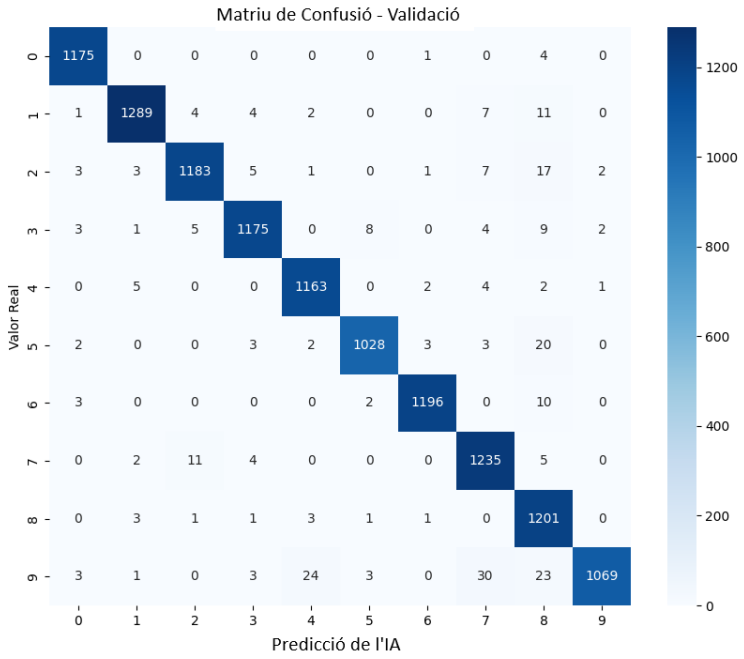


Figura 6.5: Avaluació del model CNN: Matriu de confusió en validació.

La diagonal intensament fosca de la Matriu de Confusió (Figura **??**) demostra una alta taxa d'encerts (Veritables Positius), assolint una precisió de validació propera al 98 %. La matriu de confusió mostra un bon rendiment de generalització sobre el conjunt de validació.

- **Integritat de la Transmissió (Exemples Visuals):**

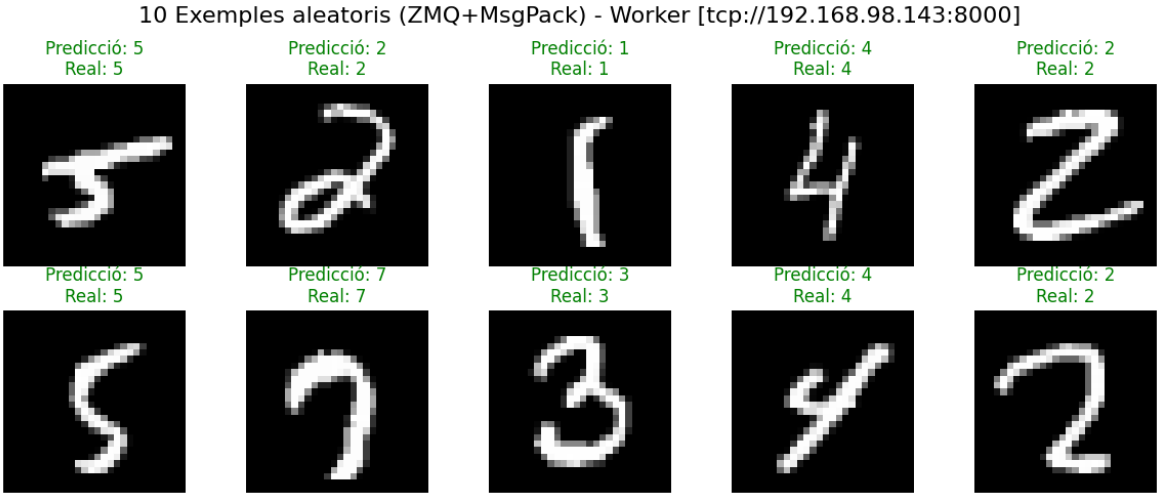


Figura 6.6: Exemples d'inferència del model d'IA: predicció i valor real al node Worker.

Les mostres aleatòries capturades directament des del *Worker* (Figura **??**) verifiquen que la serialització dinàmica amb `msgpack` reconstrueix la imatge perfecta a l'altre extrem de la xarxa, obtenint inferències lògiques i correctes.

Havent demostrat la validesa del model d'IA, aquest actua com una càrrega d'estrès realista que serveix de base per a l'avaluació principal d'infraestructures.

6.3 Comparativa Arquitectònica a l'Edge: Systemd vs Kubernetes (K3s)

Aquesta secció presenta la conclusió tècnica del projecte, enfrontant l'arquitectura nativa basada en Systemd front a l'orquestrador modern Kubernetes (K3s). Per garantir una avaluació equitativa, ambdues plataformes s'han sotmès a proves d'estrès utilitzant el protocol de xarxa més eficient identificat prèviament: **ZeroMQ amb serialització MessagePack**.

L'objectiu d'aquesta comparativa és quantificar el Sobrecost Estructural (*Architectural Overhead*) que introdueix Kubernetes al ser desplegat en hardware de recursos limitats enfront de solucions d'execució nativa.

Cal destacar que, pel que fa a l'aprovisionament (T_deploy), K3s desplega més ràpid que Systemd (145,16 s enfront de 218,67 s). Això s'explica per la capacitat del Kubelet de K3s per paral·lelitzar i gestionar eficientment l'extracció de les imatges de la seva memòria cau interna a través del containerd, en contrast amb el procés seqüencial de càrrega d'artefactes aïllats que imposa l'automatització via Ansible a l'entorn natiu.

6.3.1 L'Impost Arquitectònic (Consum de RAM en Repòs)

La primera mètrica avaluada és el consum base de la infraestructura abans d'injectar càrrega de treball, mesurat directament a nivell del sistema operatiu.

Mètrica	Systemd (Natiu)	Kubernetes (K3s)
T_deploy (Aprovisionament)	218,67 s	145,16 s
Recuperació (MTTR)	452,09 ms	1.487,28 ms
CPU Repòs	<0,1 %	0,80 %
RAM Repòs (Sobrecost Estructural)	722,50 MB	834,40 MB
Throughput d'Injecció	9.053,76 img/s	6.269,21 img/s

Taula 6.2: Comparativa global de rendiment i infraestructura: Protocol ZeroMQ-MessagePack (mitjana de 10 execucions).

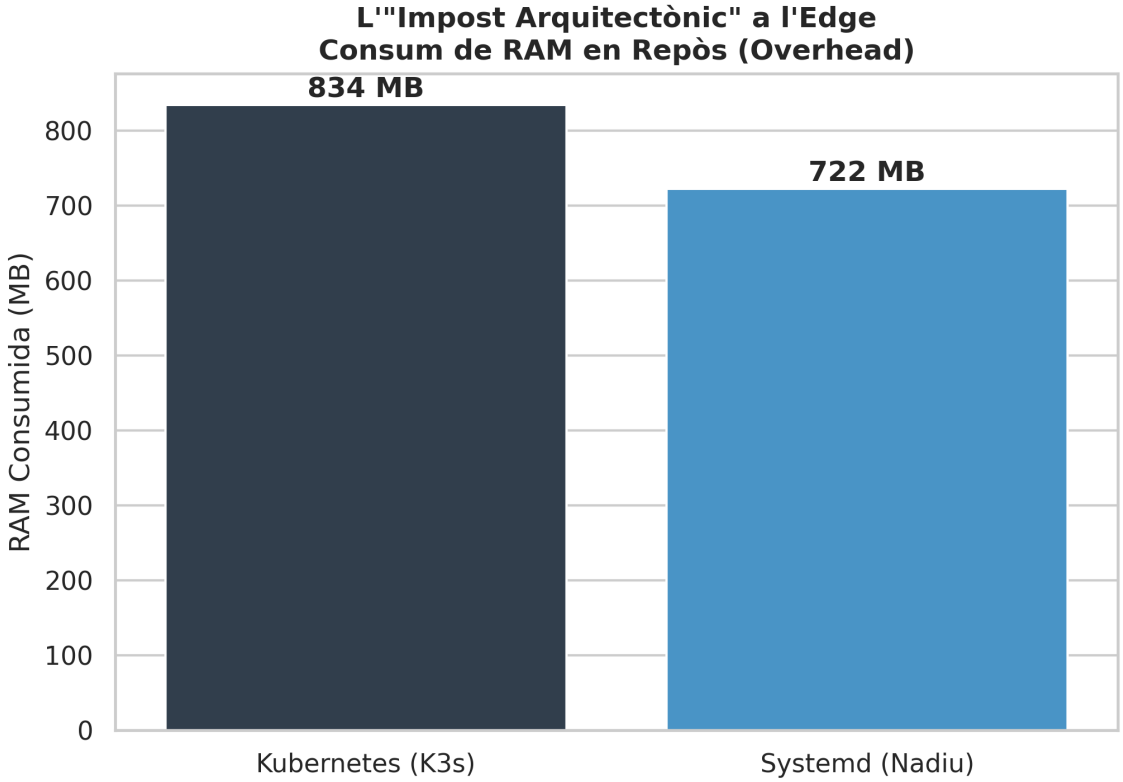


Figura 6.7: Comparativa de la petjada de memòria base (*Idle RAM Footprint*) als *Workers*.

L'arquitectura basada en Systemd presenta un consum mitjà en repòs de **722 MB**. L'adopció de Kubernetes eleva aquesta base a **834 MB** (Figura **??**). Aquesta diferència d'aproximadament **112 MB (un 15 % extra)** constitueix el sobrecost estructural directe de l'orquestrador.

Mentre que Systemd executa els contenidors de forma nativa interactuant directament amb el *kernel*, K3s requereix mantenir actius múltiples dimonis en segon pla per sostenir l'estat del clúster (*kubelet*, pla de control API, *containerd* i el controlador de xarxa *flannel*). En dispositius perimetrals amb recursos escassos, aquesta reserva permanent de memòria redueix significativament el marge disponible per al processament de càrregues d'Intel·ligència Artificial (*Compute Bound*).

6.3.2 Enginyeria del Caos i Resiliència (MTTR)

Per avaluar la capacitat d'auto-recuperació davant de fallades crítiques, es va aplicar un protocol d'*Enginyeria del Caos*, provocant l'eliminació abrupta de la unitat d'execució (`podman kill` en Sys-

temd i `kubectl delete pod --force` en K3s) i mesurant el Temps Mitjà de Recuperació (MTTR).

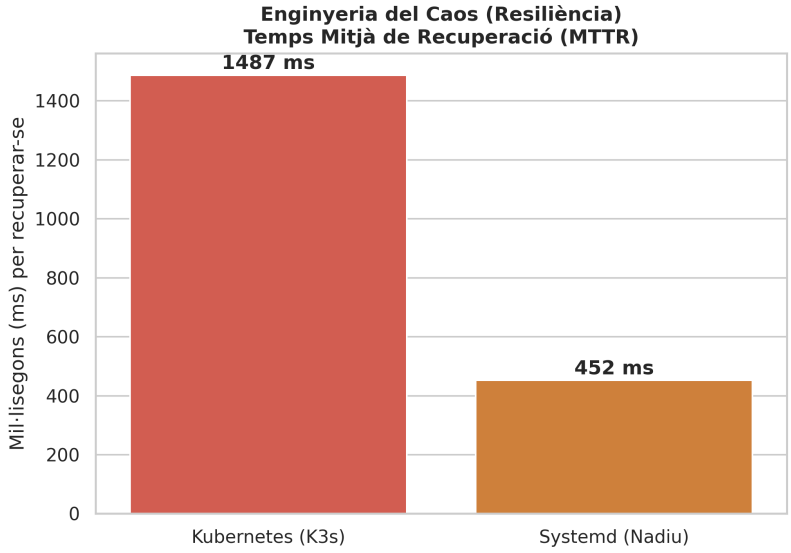


Figura 6.8: Temps Mitjà de Recuperació (MTTR) del comportament habitual davant la destrucció abrupta d'un contenidor.

Avaluant el comportament habitual de la infraestructura, Systemd demostra una velocitat de recuperació superior i altament estable, reiniciant el servei en una mitjana de **~452 ms**. Kubernetes, per la seva banda, requereix una mitjana de **~1.487 ms** (més del triple de lent), tal com reflecteix la Figura **??**. Tècnicament, Systemd gestiona els processos directament a nivell de *kernel* com a dimonis locals, la qual cosa permet un reinici gairebé instantani. Kubernetes opera sota un model distribuït asíncron: el bucle de reconciliació (*Reconciliation Loop*) ha de detectar la caiguda a través de l'API, el *Scheduler* ha de reprogramar el *Pod*, i el *Kubelet* ha d'interactuar amb el motor de contenidors i sol·licitar una nova IP virtual al CNI, afegint latència inherent al procés.

Anàlisi de Modes de Fallada Extrems (*Outliers*):

Cal destacar que en una de les execucions sobre K3s (protocol ZeroMQ-MessagePack) es va produir un outlier de 261.450 ms (més de 4 minuts), causat per un bloqueig (lock) en el connector de xarxa Flannel, que va ser incapaç d'alliberar i reassignar la IP virtual després de la destrucció del Pod.

Per facilitar la comparació del comportament habitual, es presenten a la Taula **??** les mètriques estadístiques calculades sobre les 9 execucions restants.

Tanmateix, aquest registre es conserva i s'analitza perquè evidencia un mode de fallada real i crític del sistema, propi de la complexitat estructural de les xarxes definides per programari (SDN) en entorns distribuïts.

Mètrica Estadística (Execucions Normals)	Systemd	Kubernetes (K3s)
Mitjana (*Mean*)	452,09 ms	1.487,28 ms
Mediana (*Median*)	454,76 ms	1.369,67 ms
Mínim Registrat	403,15 ms	973,70 ms
Màxim Registrat	509,38 ms	2.170,28 ms
Desviació Estàndard	±45,38 ms	±405,20 ms

Taula 6.3: Anàlisi estadístic del Temps Mitjà de Recuperació (excloent l'anomalia estructural de més de 4 minuts a K3s).

6.3.3 Penalització de Xarxa Virtual (Rendiment i Throughput)

La prova final avalua l'impacte de la capa d'infraestructura sobre el rendiment brut de la xarxa sota estrès màxim.

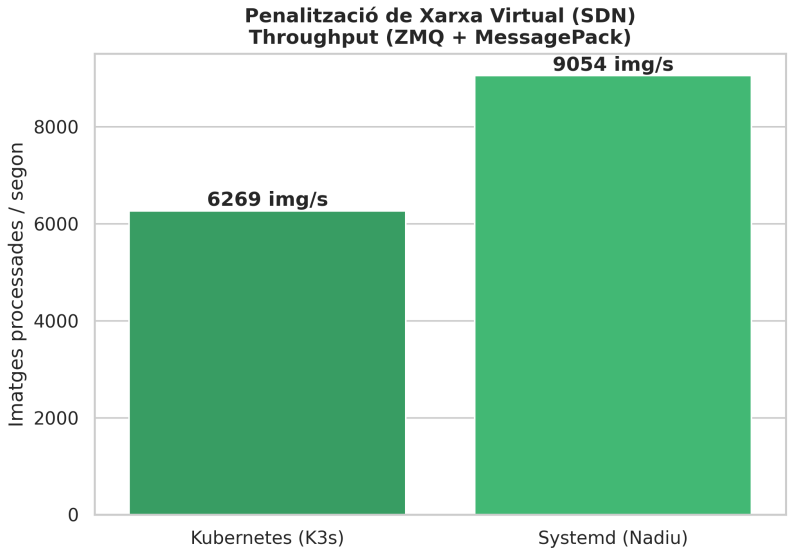


Figura 6.9: Penalització del rendiment màxim d'injecció a causa del CNI de Kubernetes.

Utilitzant exactament el mateix codi font, Systemd assoleix un processament de **~9.054 img/s**, enfront de les **~6.269 img/s** que assoleix Kubernetes (Figura **??**). Aquesta degradació del rendiment (un **31 %** inferior) és compatible amb el sobrecost introduït per la pila de xarxa de K3s, que en la configuració estudiada inclou Flannel/VXLAN i l'encaminament mitjançant NodePort. En l'arquitectura Systemd, els contenidors utilitzen una xarxa de tipus pont (*bridge*) local de Podman, evitant l'encapsulament de paquets TCP a través de túnels virtuals distribuïts i reduint la càrrega de computació sobre la CPU.

6.4 Conclusions

L'avaluació empírica conclou que portar Kubernetes a l'Edge no és "gratuït". L'orquestrador introdueix penalitzacions tangibles: incrementa la petjada de memòria en repòs al voltant d'un 15 %, triplica el temps de recuperació local davant caigudes i redueix el rendiment de xarxa en gairebé un 31 % a causa de l'encapsulació CNI.

No obstant això, la decisió arquitectònica ha de considerar la complexitat de gestió (*Management Complexity*). Systemd ofereix el màxim rendiment brut, però manca d'escalabilitat horitzontal nativa, balanceig de càrrega integrat i gestió declarativa, requerint eines externes (com Ansible) per emular un control centralitzat.

En conclusió:

- **Systemd es posiciona com l'opció més eficient entre les avaluades per a nodes Edge aïllats (*Standalone*) amb recursos altament restringits**, on el rendiment i la latència són crítics.
- **Kubernetes justifica el seu Impost Arquitectònic en escenaris on prima la reducció de la complexitat de gestió operativa**, permetent desplegaments dinàmics, tolerància a fallades a nivell de xarxa i actualitzacions declaratives (*Zero-Touch*) sobre grans flotes de dispositius distribuïts.

Capítol 7

Conclusions i Treball Futur

7.1 Assoliment d'Objectius

S'han complert íntegrament els objectius marcats a la Introducció:

- **Automatització MLOps:** S'ha implementat un flux de treball capaç d'entrenar una Xarxa Neuronal Convolucional (CNN) localment i distribuir-la mitjançant Ansible cap a nodes perimetrals basats en contenidors *rootless* (Podman).
- **Avaluació de Protocols:** S'ha demostrat empíricament la ineficiència dels estàndards web (HTTP/JSON) per a la transmissió de tensors, identificant la serialització binària dinàmica com a requisit fonamental.
- **Comparativa d'Orquestradors:** S'ha dissenyat un entorn d'Avaluació del Caos (*Chaos Testing*) automàtic que ha permès extreure mètriques precises sobre l'ús de CPU, memòria i resiliència tant en Systemd com en Kubernetes (K3s).

7.2 Contribucions Principals

L'estudi aporta un marc de decisió tècnica clar basat en dades històriques reals:

- **El paradigma ZeroMQ + MessagePack + Zero-Copy:** S'ha consolidat aquesta tríada com l'arquitectura de transport òptima a l'Edge. Elimina els colls d'ampolla del parseig de text, redueix la petjada de memòria a menys de 285 MB sota estrès màxim i permet processar més de 9.000 imatges per segon (9.054 img/s) en entorns natius.
- **Quantificació del sobrecost estructural:** S'ha mesurat el cost real de desplegar Kubernetes en dispositius limitats. K3s incrementa el consum de RAM en repòs un 15 % (~112 MB addicionals), triplica el temps de recuperació local (1.487 ms vers 452 ms) i redueix l'amplada de banda efectiva en un 31 % a causa de l'encapsulament de la seva xarxa virtual CNI (Flannel).
- **Matriu de Decisió:** Systemd es dictamina com la solució per a dispositius *Standalone* aïllats on els recursos físics són crítics. Kubernetes justifica la seva penalització de rendiment exclusivament en flotes distribuïdes on la gestió declarativa centralitzada (com els *DaemonSets*) és imperativa per a l'operació.

7.3 Limitacions Conegudes

Tot i el rigor metodològic aplicat mitjançant el registre automatitzat (Data Lake), l'estudi presenta limitacions inherents al seu disseny:

- **Homogeneïtat del Maquinari:** Les proves s'han executat en màquines virtuals basades en arquitectura x86_64 (Ubuntu). No s'ha validat l'impacte tèrmic ni els colls d'ampolla específics en arquitectures ARM pures altament restringides (ex. Raspberry Pi).
- **Topologia de Xarxa Ideal:** L'entorn opera sota una xarxa local d'alta disponibilitat (LAN). A l'Edge real (p. ex. xarxes 4G/5G industrials), factors com la pèrdua de paquets o el *jitter* extrem podrien alterar la fiabilitat de ZeroMQ.
- **Complexitat del Model d'IA:** La CNN utilitzada (MNIST) és un model d'IA lleuger. La mida dels paràmetres i el consum de memòria es dispararien exponencialment si s'utilitzessin Grans Models de Llenguatge (LLM) o xarxes de detecció d'objectes complexes.

7.4 Línies de Treball Futur

A partir d'aquesta arquitectura base, s'obren diverses vies d'investigació per optimitzar el rendiment perimetral:

- **Substitució del CNI per eBPF:** Reemplaçar Flannel per connectors de xarxa basats en eBPF (com Cilium) per eludir l'encapsulament IP i intentar reduir o eliminar la penalització d'aproximadament el 31 % de rendiment a K3s.
- **Integració d'Acceleradors Maquinari (GPUs/NPUs):** Ampliar els *playbooks* d'Ansible per exposar directament acceleradors físics cap als contenidors Podman i K3s, avaluant com l'orquestració afecta la latència en operacions tensorials pures.
- **Inclusió de *Brokers* Lleugers a l'Edge:** Substituir l'arquitectura P2P (*Request-Reply*) per un model de publicació-subscripció utilitzant *brokers* d'alt rendiment (com NATS o MQTT) per testejar topologies on múltiples orígens de dades (sensors IoT) alimenten el node d'inferència.